AdaptSFL: Adaptive Split Federated Learning in Resource-Constrained Edge Networks

Zheng Lin, Guanqiao Qu, *Graduate Student Member, IEEE*, Wei Wei, Xianhao Chen, *Member, IEEE*, and Kin K. Leung, *Life Fellow, IEEE*

***Abstract*—The increasing complexity of deep neural networks poses significant barriers to democratizing AI to resource-limited edge devices. To address this challenge, split federated learning (SFL) has emerged as a promising solution that enables device-server co-training through model splitting. However, although system optimization substantially influences the performance of SFL, the problem remains largely uncharted. In this paper, we first provide a unified convergence analysis of SFL, which quantifies the impact of model splitting (MS) and client-side model aggregation (MA) on its learning performance, laying a theoretical foundation for this field. Based on this convergence bound, we introduce AdaptSFL, an adaptive SFL framework to accelerate SFL under resource-constrained edge computing systems. Specifically, AdaptSFL adaptively controls MS and client-side MA to balance communication-computing latency and training convergence. Extensive simulations across various datasets validate that our proposed AdaptSFL framework takes considerably less time to achieve target accuracy than existing benchmarks.**

***Index Terms*—Distributed learning, split federated learning, client-side model aggregation, model splitting, mobile edge computing.**

I. Introduction

With the increasing prevalence of edge devices such as smartphones, laptops, and wearable gadgets, a vast amount of data is being generated at the network edge. It is forecasted that there will be 50 billion IoT devices, generating 79.4 Zettabytes of data in 2025 [1]. This unprecedented volume of data unlocks the potential of artificial intelligence (AI), leading to significant advancements in pivotal domains such as smart health, intelligent transportation, and natural language processing [2], [3], [4], [5], [6], [7].

Status quo model training frameworks rely on centralized learning (CL), where a cloud server gathers raw data from participating devices to train a machine learning (ML) model. However, transferring enormous amounts of data from edge devices to the cloud server leads to significant latency, huge bandwidth costs, and severe privacy leakage [8], [9], [10], [11]. To tackle this issue, federated learning (FL) [12], [13] has been proposed as the answer for privacy-enhancing distributed training. In FL, all participating devices perform model training with locally residing data in parallel and synchronize their models by exchanging model parameters rather than raw data with a parameter server (or fed server), thereby enhancing data privacy [14], [15]. Unfortunately, while FL overcomes the limitations of CL, implementing FL poses a significant challenge as ML models scale up [16]. For instance, the recently prevalent Gemini Nano-2 model, an on-device large language model, comprises 3.25 billion parameters (3GB for 32-bit floats), making on-device training, as required by FL, extremely difficult for resource-constrained edge devices [17].

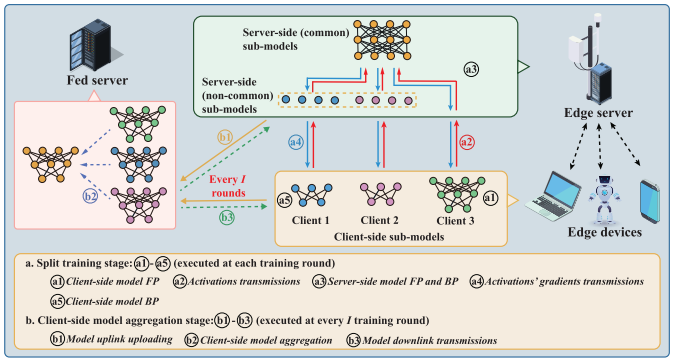


Fig. 1. The illustration of AdaptSFL over edge computing systems.

To mitigate the dilemma in FL, split learning (SL) [18] has emerged as an alternative privacy-enhancing distributed training paradigm, which enables a server to take over the major workload from clients via model splitting, thus relieving the client-side computing burden [19], [20]. Moreover, since each client sends intermediate results at the cut layer instead of raw data or entire ML model parameters, SL preserves data privacy while reducing communication overhead compared to FL if the model size is large [19]. Despite these advantages, the original SL, named vanilla SL [18], adopts a sequential training procedure from one device to another, resulting in excessive training time. As a variant of SL, split federated learning (SFL) [21] overcomes this issue by enabling parallel training among edge devices. As shown in Fig. 1, apart from model splitting, SFL also features periodic server-side and client-side sub-model aggregation following the principle of FL. By amalgamating the advantages of SL and FL, SFL

Received 19 March 2024; revised 28 December 2024 and 28 March 2025; accepted 4 June 2025; approved by IEEE Transactions on Networking Editor C. Joe-Wong. Date of publication 25 June 2025; date of current version 18 December 2025. This work was supported in part by the Research Grants Council of Hong Kong under Grant 27213824 and Grant Collaborative Research Scheme (CRS) HKU702/24 and in part by the University of Hong Kong (HKU) Institute of Data Science (IDS) Research Seed Fund under Grant IDS-RSF2023-0012. The work of Kin K. Leung was supported in part by Engineering and Physical Sciences Research Council (EPSRC) under Grant EP/Y037242/1. *(Corresponding author: Xianhao Chen.)*

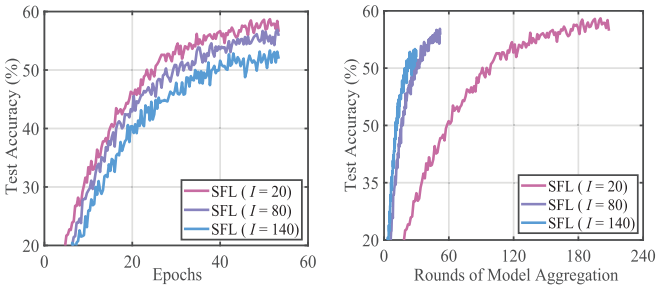
Zheng Lin, Guanqiao Qu, Wei Wei, and Xianhao Chen are with the Department of Electrical and Electronic Engineering, The University of Hong Kong, Hong Kong, China (e-mail: linzheng@eee.hku.hk; gqqu@eee.hku.hk; weiwei@eee.hku.hk; xchen@eee.hku.hk).

Kin K. Leung is with the Department of Electrical and Electronic Engineering and the Department of Computing, Imperial College, SW7 2BT London, U.K. (e-mail: kin.leung@imperial.ac.uk).

Digital Object Identifier 10.1109/TON.2025.3577790

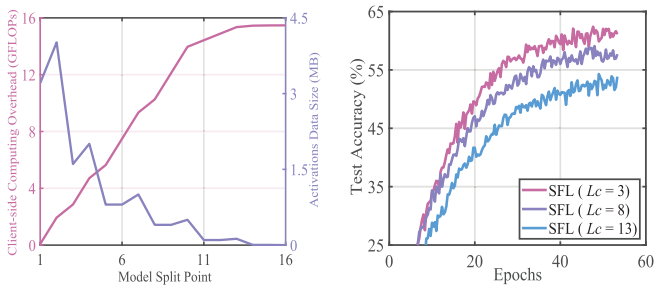
2998-4157 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.



(a) Test accuracy versus epochs. (b) Test accuracy versus rounds of model aggregation.

Fig. 2. The impact of client-side MA on training performance and communication overhead. Fig. 2(a) and Fig. 2(b) show the performance for test accuracy versus number of epochs and rounds of model aggregation. The experiment is conducted on the CIFAR-10 dataset under the non-IID setting with $L_c = 8$.



(a) Computing and communication overhead. (b) Test accuracy versus epochs.

Fig. 3. The impact of MS on computing and communication overhead as well as training performance. Fig. 3(a) shows the computing and communication overhead of SFL at different model split points. Fig. 3(b) presents the performance for test accuracy versus the number of epochs. The experiment is conducted on the CIFAR-10 dataset under the non-IID setting with $I = 15$.

has attracted significant attention from academia and industry in recent years. For instance, Huawei deems SFL as a key learning framework for 6G edge intelligence [22].

While SFL, akin to any other learning framework, demands intensive communication-computing resources, system optimization for SFL is still in its nascent stage. In particular, model splitting (MS) and client-side model aggregation (MA) decisions heavily influence model accuracy and total training latency of SFL in resource-constrained edge networks. Although many prior works [23], [24], [25] have made efforts to optimize global model aggregation frequencies in FL, SFL exhibits entirely different convergence behavior because server-side sub-models and client-side sub-models in SFL can be aggregated at *varied frequencies*, making convergence analysis and optimization non-trivial. Specifically, server-side sub-models, which are co-located on a server, should *always be synchronized* without communication costs,[1] whereas client-side sub-model aggregation generates substantial communication traffic from edge devices, and therefore, should be performed *less frequently*. To demonstrate the impact of client-side MA, Fig. 2 shows that aggregation interval I (client-side sub-models are aggregated every I training rounds) balances the converged accuracy and communication overhead: A smaller I usually leads to higher converged accuracy while incurring more communication costs, implying client-side MA should be carefully controlled. On the other hand, MS also plays a vital role in SFL as it not only affects the computing and communication workloads but also directly impacts the convergence speed of the training process. To show the effect of MS, Fig. 3 reveals that a shallow cut layer (i.e., smaller L_c) enhances training accuracy since a larger portion of the model serves as the server-side sub-model that can be synchronized in every training round. However, this may result in increased communication overhead for transmitting cut-layer activations/gradients because a shallower layer in CNNs often involves a larger dimension of layer output. Overall, optimizing MS and client-side MA requires fundamental tradeoff analysis to accelerate SFL under communication and computing constraints.

To fill the above void, this paper aims to minimize the training latency of SFL for achieving target learning performance under resource-constrained edge computing systems. To this end, we propose AdaptSFL, a novel resource-adaptive SFL framework with adaptive client-side MA and MS under communication-computing resource constraints. To establish the foundation, we first derive a convergence bound of AdaptSFL that characterizes the effect of client-side MA and MS on its training convergence. This derivation differs from the analysis of FL because client-side and server-side sub-models in SFL can be aggregated at different frequencies. Based on this, we formulate and solve the optimization problem of MS and client-side MA to minimize the total latency under training loss and resource constraints. The main contributions of this paper are summarized as follows.

- We investigate the system optimization for SFL over resource-constrained edge systems. We theoretically analyze the convergence bound of AdaptSFL, providing some intriguing insights that can lay the foundation for SFL optimization. To our best knowledge, this is the first convergence analysis of SFL that characterizes the effect of MS and client-side MA on its training convergence.
- We leverage the derived convergence bound to formulate a joint optimization problem for client-side MA and MS to minimize the total training time for model convergence under resource-constrained edge computing systems.
- We decompose the optimization problem into two tractable subproblems of client-side MA and MS, and develop efficient algorithms to obtain the optimal solutions for them, respectively. Subsequently, we develop an alternating optimization method to obtain an efficient suboptimal solution from the joint problem.
- We conduct extensive simulations across various datasets to validate our theoretical analysis and demonstrate the superiority of the proposed solution over the counterpart without optimization.

The remainder of this paper is organized as follows. Section II introduces related work. Section III elaborates on system model and AdaptSFL framework. Section IV provides the convergence analysis. We formulate the optimization problem in Section V and offer the corresponding solution approach in Section VI. Section VII provides the simulation results. Finally, concluding remarks are presented in Section VIII.

II. Related Work

Tremendous research efforts have been made to improve the learning performance of distributed ML by controlling

[1]This is because more frequent model aggregation generally leads to better training convergence as demonstrated by both theoretical analysis and experiments [25].

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.

the interval of model aggregation [12], [25], [26], [27], [28], [29]. The design often lies in convergence analysis, which serves as the basis for optimizing model aggregation. McMahan et al. [12] develop a pioneering FedAvg algorithm that enables each participant to conduct multiple local stochastic gradient descent iterations in a communication round, which aims to reduce the global synchronization frequency without considerably compromising training accuracy. Wang et al. [25] propose a control algorithm that determines the best tradeoff between local update and global parameter aggregation to minimize the loss function under a given resource budget. Wu et al. [26] devise an adaptive momentum-based FL algorithm to achieve the optimal MA interval, which lowers the time consumed for achieving the target accuracy. Smith et al. [27] and Yu and Jin [28] empirically design dynamic batch size schemes to reduce MA interval to establish a more communication-efficient FL framework. Xiang et al. [29] propose an adaptive epoch adjustment method to reduce communication rounds by dynamically adjusting MA interval of end devices. Nevertheless, the aforementioned schemes do not apply to SFL as server-side sub-models, which are co-located at the same place, can always be synchronized in SFL.

On the other hand, MS heavily impacts training latency of SL, as it leads to varied computing workloads between devices and the server, varied communication overhead due to the diverse output data sizes across layers, and varied convergence speed dependent on the proportion of the server-side common model. Several studies have delved into determining the optimal model split point for SL [30], [31], [32]. Wu et al. [30] develop a cluster-based parallel SL framework and a joint cut layer selection, device clustering, and subchannel allocation optimization algorithm to minimize the training overhead considering device heterogeneity and network dynamics. Wang et al. [31] present a time-efficient multi-split ML framework, which enables real-time optimal split decisions among multiple computing nodes and dynamically adapts split decisions in response to instantaneous network dynamics. Lin et al. [32] propose an efficient parallel SL framework featuring last-layer gradient aggregation, lowering the dimension of activations' gradients and server-side computing workload. Based on this framework, a joint resource management and MS strategy has been proposed to jointly optimize subchannel allocation, power control, and cut layer selection to minimize the training latency. Unfortunately, these works fail to characterize the effect of MS on training convergence.

In a nutshell, the problem of optimizing MA and MS to enhance SFL convergence has not been investigated. The problem is non-trivial for two reasons: First, a unified convergence analysis that quantifies the impact of client-side MA and MS on the convergence of SFL is still lacking. Second, considering the heterogeneous bandwidth and computing resources of edge devices, MS and client-side MA need to be jointly optimized to accelerate training convergence, resulting in a challenging optimization problem.

In addition to MS and MA optimization, it is worth mentioning that there are research efforts to improve the resource efficiency of SFL from other perspectives. For instance, Pal et al. [33] propose a server-side local gradient averaging mechanism and adaptive learning rate acceleration to enhance the scalability and efficiency of SL. Liao et al. [34] mitigate system heterogeneity by clustering workers for split training and assigning different local update frequencies to each cluster, thus improving training efficiency and model accuracy. Guo et al. Guo et al. [35] design a selective parameter transmission method for model aggregation while dynamically reconstructing the global model to enhance communication efficiency and data privacy. However, these innovative frameworks often lack theoretical convergence guarantees. In contrast, our scheme focuses on vanilla SFL and provides a fundamental trade-off analysis of MA and MS under resource-constrained edge networks.

III. The Resource-Adaptive SFL Framework

In this section, we present the system model in Section III-A and the proposed resource-adaptive SFL framework, named AdaptSFL, in Section III-B.

A. System Model

In this section, we introduce the system model of AdaptSFL to provide a theoretical foundation for the following sections. As illustrated in Fig. 1, we consider a typical scenario of AdaptSFL over edge networks, which consists of three fundamental components:

- **Edge devices:** We consider that each client possesses an edge device with computing capability to execute the client-side forward propagation (FP) and back-propagation (BP). The set of participating edge devices is denoted by $\mathcal{N} = \{1, 2, \ldots, N\}$, where N is the number of edge devices. Each edge device i has its local dataset $\mathcal{D}_i = \{\mathbf{x}_{i,k}, y_{i,k}\}_{k=1}^{D_i}$ with D_i data samples, where $\mathbf{x}_{i,k}$ and $y_{i,k}$ are the k-th input data and its corresponding label. The client-side sub-model of edge device i is represented as $\mathbf{w}_{c,i}$.
- **Edge server:** The edge server is a powerful central computing entity responsible for performing server-side sub-model training. The server-side sub-model of edge device i is denoted by $\mathbf{w}_{s,i} = [\mathbf{h}_s; \mathbf{h}_{m,i}]$, where $\mathbf{h}_s$ and $\mathbf{h}_{m,i}$ denote the server-side common and server-side non-common sub-models, respectively. The common sub-model means it is shared by all clients and synchronized in every round. The non-common sub-model stems from the fact that some clients have more layers than others on the server, which is aggregated every I rounds together with the client-side sub-models. Furthermore, the edge server also takes charge of collecting important network information, such as the device computing capabilities and channel states, to support optimization decisions.
- **Fed server:** The fed server takes charge of client-side sub-model synchronization, periodically aggregating the client-side sub-models from all participating edge devices. For privacy concerns, fed and edge servers usually do not belong to the identical party because a malicious server can recover the raw data if getting both client-side sub-models and smashed data [36].

The global model is denoted by $\mathbf{w} = [\mathbf{w}_{s,i}; \mathbf{w}_{c,i}]$. The objective of SL is to find the optimal global model $\mathbf{w}^*$ that achieves good performance across all participating devices, which can be formulated as minimizing a finite-sum non-convex global loss function:

$$\min_{\mathbf{w}} f(\mathbf{w}) \triangleq \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^{N} f_i(\mathbf{w}), \tag{1}$$

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SUMMARY OF IMPORTANT NOTATIONS

Notation	Description
$\mathcal{N}$	The set of participating edge devices
$\mathcal{D}_i$	The local dataset of edge device i
$f(\mathbf{w})$	The global loss function with parameter $\mathbf{w}$
$\mathcal{R}$	The set of training rounds
$\mathcal{B}_i$	The mini-batch drawn from edge device i's local dataset
b	The mini-batch size
$\mathbf{w}_{c,i}/\mathbf{w}_{s,i}$	The client-side/server-side sub-model of edge device i
$\mathbf{h}_s$	The server-side common sub-model
$\mathbf{h}_{m,i}$	The server-side non-common sub-model of edge device i
$\mathbf{h}_{c,i}$	The forged client-specific model of edge device i
L_c	The maximum number of layers of client-side sub-models
L	The total number of global model layers
f_s/f_i	The computing capability of server/edge device i
ρ_j/ϖ_j	The FP/BP computing workload of propagating the first j layer neural network for one data sample
ψ_j/χ_j	The data size of the activations/activations' gradients at the cut layer j
δ_j	The data size of client-side sub-model with the cut layer j
$r_i^U/r_{i,f}^U$	The uplink transmission rate from edge device i to edge server/fed server.
$r_i^D/r_{i,f}^D$	The downlink transmission rate from edge server/ fed server to edge device i.
$r_{s,f}$	The transmission rate from the edge server to fed server.
$r_{f,s}$	The transmission rate from fed server to edge server.
β, σ_j^2, G_j^2	The loss function constants (detailed in Section IV)
I, $\mu_{i,j}$	The decision variables (explained in Section V)
T_1-T_6	The auxiliary variables (to linearize problem (28))

where $f_i(\mathbf{w})$ represents the local loss function of edge device i over local dataset $\mathcal{D}_i$, i.e., $f_i(\mathbf{w}) \triangleq \mathbb{E}_{\xi_i \sim \mathcal{D}_i}[F_i(\mathbf{w};\xi_i)]$, and ξ_i is training randomness from local dataset $\mathcal{D}_i$, i.e., stochasticity of data order during training caused by the sampling of mini-batch data. We consider uniformity in the sizes of local datasets across all edge devices for concise expression. Following the standard stochastic optimization setting [37], [38], [39], it is assumed that the stochastic gradient is an unbiased estimator of the true gradient, i.e., $\mathbb{E}_{\xi_i^t \sim \mathcal{D}_i}[\nabla F_i(\mathbf{w}_i^{t-1};\xi_i^t)|\boldsymbol{\xi}^{[t-1]}] = \nabla f_i(\mathbf{w}_i^{t-1})$, where $\boldsymbol{\xi}^{[t-1]} \triangleq [\xi_i^\tau]_{i\in\{1,2,\ldots,N\},\tau\in\{1,\ldots,t-1\}}$ denotes all the randomness up to training round $t-1$.

To solve problem (1), the conventional SFL framework aggregates both client-side and server-side sub-models every training round. However, aggregating client-side sub-models at each training round leads to communication inefficiency, consuming considerable communication resources, especially for implementing SFL over mobile edge devices. Moreover, the MS has not been studied by jointly considering its impact on training convergence and communication-computing workload. Motivated by this, we propose a state-of-the-art AdaptSFL framework capable of resource-adaptive client-side MA and MS. For readers' convenience, the important notations in this paper are summarized in Table I.

B. The AdaptSFL Framework

This section details the workflow of the proposed AdaptSFL framework. The distinctive feature of AdaptSFL lies in resource-adaptive client-side MA and MS. By optimizing the client-side MA and MS, training latency can be significantly reduced for achieving target learning performance.

Before model training begins, the edge server initializes the ML model, partitions it into client-side and server-side sub-models via layer-wise MS (shown in Section VI), and determines the optimal client-side MA (discussed in Section VI). Afterward, AdaptSFL is executed in I consecutive training rounds. This process loops until the model converges. The training process of AdaptSFL involves two primary stages, i.e., split training and client-side MA. The split training is performed in each training round, while the client-side MA occurs every I training round. As depicted in Fig. 1, for a training round $t \in \mathcal{R} = \{1, 2, \ldots, R\}$, the training process of AdaptSFL is detailed as follows.

a. Split Training Stage: The split training stage involves model updates for participating edge devices and an edge server every training round. This stage comprises the following five steps.

a1) Client-side Model Forward Propagation: In this step, all participating edge devices execute client-side FP in parallel. To be specific, edge device i randomly draws a mini-batch $\mathcal{B}_i \subseteq \mathcal{D}_i$ with b data samples from its local dataset for model training. The input data and corresponding label of mini-batch in training round t are denoted by $\mathbf{x}_i^t$ and $\mathbf{y}_i^t$, and $\mathbf{w}_{c,i}^{t-1}$ represents the client-side sub-model of edge device i. After feeding a mini-batch of data into the client-side sub-model, activations are generated at the cut layer. The activations of edge device i are represented as

$$\mathbf{a}_i^t = \varphi\left(\mathbf{x}_i^t; \mathbf{w}_{c,i}^{t-1}\right), \tag{2}$$

where $\varphi(x; w)$ maps the relationship between input data x and its predicted value given model parameter w.

a2) Activations Transmissions: After completing client-side FP, each edge device sends its respective activations and labels to the edge server (usually over wireless channels). The collected activations from participating edge devices are then utilized to fuel server-side model training.

a3) Server-side Model Forward Propagation and Back-propagation: After receiving activations from participating edge devices, the edge server feeds these activations into the server-side sub-models to execute the server-side FP. For edge device i, the predicted value is expressed as

$$\hat{\mathbf{y}}_i^t = \varphi\left(\mathbf{a}_i^t; \mathbf{w}_{s,i}^{t-1}\right), \tag{3}$$

where $\mathbf{w}_{s,i}^{t-1} = \left[\mathbf{h}_s^{t-1}; \mathbf{h}_{m,i}^{t-1}\right]$, $\mathbf{h}_s^{t-1}$ and $\mathbf{h}_{m,i}^{t-1}$ denote the server-side common and server-side non-common sub-models, respectively. The predicted value and labels are utilized to calculate the loss function value and further derive the server-side sub-model's gradients.

For the server-side common sub-model, the server updates it[2] based on

$$\mathbf{h}_s^t = \frac{1}{N}\sum_{i=1}^{N} \mathbf{h}_{s,i}^t, \tag{4}$$

where $\mathbf{h}_{s,i}^t \leftarrow \mathbf{h}_{s,i}^{t-1} - \gamma \nabla_{\mathbf{h}_s} F_i(\mathbf{h}_{s,i}^{t-1}; \xi_i^t)$ is server-side common sub-model for edge device i and $\nabla_{\mathbf{w}} F(\mathbf{w};\xi)$ represents the gradient of the function $F(\mathbf{w};\xi)$ with respect to the model parameter $\mathbf{w}$. Since the aggregation in Eqn. (4) does not incur any communication overhead, we assume it is conducted every

[2]The edge server can execute model update for multiple edge devices in either a serial or parallel fashion, which does not affect training performance. Here, we consider the parallel fashion.

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.

training round to improve training convergence. In this way, the server-side common sub-model is equivalent to centralized training (i.e., stochastic gradient descent or SGD).

Due to the heterogeneous cut layers, some edge devices can leave more layers on the server side, resulting in server-side non-common parts of server-side sub-models. Each edge device updates its respective server-side non-common sub-model individually, i.e.,

$$\mathbf{h}_{m,i}^{t} \leftarrow \mathbf{h}_{m,i}^{t-1} - \gamma \nabla_{\mathbf{h}_m} F_i(\mathbf{h}_{m,i}^{t-1}; \xi_i^t), \tag{5}$$

where $\nabla_{\mathbf{h}_s} F_i(\mathbf{h}_s^{t-1}; \xi_i^t)$ and $\nabla_{\mathbf{h}_m} F_i(\mathbf{h}_{m,i}^{t-1}; \xi_i^t)$ represent the stochastic gradients of server-side common and non-common sub-models of edge device *i*, and γ is the learning rate.

a4) Activations' Gradients Transmissions: After the server-side BP is completed, the edge server transmits the activations' gradients to its corresponding edge devices.

a5) Client-side Model Back-propagation: In this step, each edge device updates its client-side sub-model parameters based on the received activations' gradients. For edge device *i*, the client-side sub-model is updated through

$$\mathbf{w}_{c,i}^{t} \leftarrow \mathbf{w}_{c,i}^{t-1} - \gamma \nabla_{\mathbf{w}_c} F_i(\mathbf{w}_{c,i}^{t-1}; \xi_i^t). \tag{6}$$

b. Client-side Model Aggregation Stage: The client-side MA stage primarily focuses on aggregating forged client-specific sub-models (including client-side sub-models and server-side non-common sub-models specific to each client) on the fed server, which is executed every *I* training round. This stage consists of the following three steps.

b1) Model Uplink Uploading: In this step, edge devices and edge server simultaneously send the client-side sub-models and server-side non-common sub-models to the fed server over the wireless/wired links.

b2) Client-side Model Aggregation: The fed server pairs and assembles the received client-side sub-models and server-side non-common sub-models to forge the client-specific models. Afterwards, these forged client-specific models are aggregated, denoted by

$$\mathbf{h}_c^t = \frac{1}{N} \sum_{i=1}^{N} \mathbf{h}_{c,i}^t, \tag{7}$$

where $\mathbf{h}_{c,i}^t = \left[\mathbf{h}_{m,i}^t; \mathbf{w}_{c,i}^t\right]$ is forged client-specific model of edge device *i*.

b3) Model Downlink Transmissions: After completing the client-side MA, the fed server sends updated client-side sub-models and server-side non-common sub-models to corresponding edge devices and the edge server, respectively.

The AdaptSFL training framework is outlined in Algorithm 1, where *E* is the total number of stochastic gradients involved in model training.

Algorithm 1 The AdaptSFL Training Framework

Input: b, γ, E, $\mathcal{N}$ and $\mathcal{D}$.
Output: $\mathbf{w}^*$.

Initialization: $\mathbf{w}^0$, $\mathbf{w}_i^0 \leftarrow \mathbf{w}^0$, $I^0 \leftarrow 1$, $\tau \leftarrow 0$ and $\rho \leftarrow 0$.
while $b \sum_{\eta=0}^{\tau} I^\eta \leq E$ **do**
  Determine I^τ and $\boldsymbol{\mu}^\tau$ based on Algorithm 3
  for $t = \rho+1, \rho+2, \ldots, \rho+I^\tau$ **do**

    /** Runs on edge devices **/
    for all edge device $i \in \mathcal{N}$ in parallel **do**
      $\mathbf{a}_i^t \leftarrow \varphi\left(\mathbf{x}_i^t; \mathbf{w}_{c,i}^{t-1}\right)$
      Send $(\mathbf{a}_i^t, \mathbf{y}_i^t)$ to the edge server
    end for

    /** Runs on edge server **/
    $\hat{\mathbf{y}}_i^t = \varphi\left(\mathbf{a}_i^t; \mathbf{w}_{s,i}^{t-1}\right)$
    Calculate loss function value $f\left(\mathbf{w}^{t-1}\right)$
    $\mathbf{h}_s^t \leftarrow \mathbf{h}_s^{t-1} - \gamma \frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{h}_s} F_i(\mathbf{h}_s^{t-1}; \xi_i^t)$,
    $\mathbf{h}_{m,i}^t \leftarrow \mathbf{h}_{m,i}^{t-1} - \gamma \nabla_{\mathbf{h}_m} F_i(\mathbf{h}_{m,i}^{t-1}; \xi_i^t)$
    Send activations' gradients to corresponding edge devices

    /** Runs on edge devices **/
    for all edge device $i \in \mathcal{N}$ in parallel **do**
      $\mathbf{w}_{c,i}^t \leftarrow \mathbf{w}_{c,i}^{t-1} - \gamma \nabla_{\mathbf{w}_c} F_i(\mathbf{w}_{c,i}^{t-1}; \xi_i^t)$
    end for

    /** Runs on the fed server **/
    if $(t-\rho) \bmod I^\tau = 0$ **then**
      Forge client-side sub-models $\mathbf{h}_{c,i}^t = \left[\mathbf{h}_{m,i}^t; \mathbf{w}_{c,i}^t\right]$
      $\mathbf{h}_c^t = \frac{1}{N} \sum_{i=1}^{N} \mathbf{h}_{c,i}^t$
      $\mathbf{h}_{c,i}^t \leftarrow \mathbf{h}_c^t$
    end if
  end for

  Update $\rho \leftarrow \rho + I^\tau$
  Update $\tau \leftarrow \tau + 1$
end while

IV. Convergence Analysis of AdaptSFL

In this section, we provide the first convergence analysis of SFL that characterizes the effect of both client-side MA and MS on its training convergence, which serves as the basis for developing an efficient iterative method in Section VI.

Before analysis, we first clarify several notations to simplify the derivation process and results. Considering the variance in MA interval, we define the global model of edge device *i* as $\mathbf{w}_i^t = \left[\mathbf{h}_{s,i}^t; \mathbf{h}_{c,i}^t\right]$, which contains server-side common sub-model and forged client-specific model. The server-side common sub-models are aggregated each training round, while the client-specific model is aggregated every *I* training round. The model split point between these two sub-models is L_c, defined as the maximum number of layers of client-side sub-models due to discrepancies in model split points among different edge devices. The corresponding gradients of these two sub-models are determined as

$$\mathbf{g}_{c,i}^t = \left[\nabla_{\mathbf{h}_m} F_i(\mathbf{h}_{m,i}^{t-1}; \xi_i^t); \nabla_{\mathbf{w}_c} F_i(\mathbf{w}_{c,i}^{t-1}; \xi_i^t)\right] \tag{8}$$

and

$$\mathbf{g}_{s,i}^t = \sum_{i=1}^{N} \nabla_{\mathbf{h}_s} F_i(\mathbf{h}_s^{t-1}; \xi_i^t), \tag{9}$$

where $\mathbf{h}_s^t = \frac{1}{N} \sum_{i=1}^{N} \mathbf{h}_{s,i}^t$. Therefore, the gradient of the global model is represented as $\mathbf{g}_i^t = \left[\mathbf{g}_{s,i}^t; \mathbf{g}_{c,i}^t\right]$.

In line with the seminal works in distributed stochastic optimization [40], [41], [42], [43] where convergence analysis

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.

is conducted on an aggregated version of individual solutions, this paper focuses on the convergence analysis of $\mathbf{w}^t = \frac{1}{N}\sum_{i=1}^{N}\mathbf{w}_i^t$. To analyze the convergence rate of Algorithm 1, we consider the following two standard assumptions on the loss functions.

Assumption 1 (Smoothness): Each local loss function $f_i(\mathbf{w})$ is differentiable and β-smooth., i.e., for any $\mathbf{w}$ and $\mathbf{w}'$, we have

$$\|\nabla_{\mathbf{w}} f_i(\mathbf{w}) - \nabla_{\mathbf{w}} f_i(\mathbf{w}')\| \leq \beta \|\mathbf{w} - \mathbf{w}'\|, \ \forall i. \tag{10}$$

Assumption 2 (Bounded Variances and Second Moments): The variance and second moments of stochastic gradients for each layer have upper bound:

$$\mathbb{E}_{\xi_i \sim \mathcal{D}_i}\|\nabla_{\mathbf{w}} F_i(\mathbf{w};\xi_i) - \nabla_{\mathbf{w}} f_i(\mathbf{w})\|^2 \leq \sum_{j=1}^{l}\sigma_j^2, \forall \mathbf{w}, \ \forall i. \tag{11}$$

$$\mathbb{E}_{\xi_i \sim \mathcal{D}_i}\|\nabla_{\mathbf{w}} F_i(\mathbf{w};\xi_i)\|^2 \leq \sum_{j=1}^{l} G_j^2, \forall \mathbf{w}, \ \forall i, \tag{12}$$

where l is number of layers for model $\mathbf{w}$, σ_j^2 and G_j^2 denote the bounded variance and second order moments for the j-th layer of model $\mathbf{w}$.

Lemma 1: Under Assumption 1, Algorithm 1 ensures

$$\mathbb{E}[\|\mathbf{h}_c^t - \mathbf{h}_{c,i}^t\|^2] \leq 1_{\{I>1\}} 4\gamma^2 I^2 \sum_{j=1}^{L_c} G_j^2, \forall i, \forall t,$$

where I denotes client-side model aggregation interval and $\mathbf{h}_c^t$ is defined in Eqn. (7).

Proof: See Appendix A. □

Theorem 1: Under Assumption 1 and Assumption 2, if $0 < \gamma \leq \frac{1}{\beta}$ in Algorithm 1, then for all $R \geq 1$, we have

$$\frac{1}{R}\sum_{t=1}^{R}\mathbb{E}[\|\nabla_{\mathbf{w}} f(\mathbf{w}^{t-1})\|^2] \leq \frac{2\vartheta}{\gamma R} + \frac{\beta\gamma\sum_{j=1}^{L}\sigma_j^2}{N} + 1_{\{I>1\}} 4\beta^2\gamma^2 I^2 \sum_{j=1}^{L_c} G_j^2, \tag{13}$$

where $\vartheta = f(\mathbf{w}^0) - f^*$, L and f^* represent the total number of global model layers and minimum value of problem (1).

Proof: See Appendix B. □

Remark: Compared to conventional FL [12], [25], [26], the convergence bound of AdaptSFL exhibits two key differences. First, the varied client-side and server-side MA intervals introduce an additional error term proportional to I^2, as client-side updates are aggregated every I rounds, causing model staleness and slowing model convergence. Second, the MS decision variable L_c governs the synchronization efficiency: a smaller L_c increases the proportion of the model updated each round, improving convergence speed, whereas a larger L_c degrades convergence performance.

Substituting Eqn. (13) into Eqn. (14) yields Corollary 1, revealing a lower bound on the number of training rounds for achieving target convergence accuracy.

Corollary 1: The number of training rounds R for achieving target convergence accuracy ε, i.e., satisfying

$$\frac{1}{R}\sum_{t=1}^{R}\mathbb{E}[\|\nabla_{\mathbf{w}} f(\mathbf{w}^{t-1})\|^2] \leq \varepsilon, \tag{14}$$

is given by

$$R \geq \frac{2\vartheta}{\gamma\left(\varepsilon - \frac{\beta\gamma\sum_{j=1}^{L}\sigma_j^2}{N} - 1_{\{I>1\}} 4\beta^2\gamma^2 I^2 \sum_{j=1}^{L_c} G_j^2\right)}. \tag{15}$$

Insights: Eqn. (15) shows that the number of training rounds for achieving target convergence accuracy ε decreases as I decreases and the cut layer becomes shallow, indicating a faster model convergence rate. Similarly, for a given training round R, more frequent client-side MA and a shallower cut layer result in higher converged accuracy (i.e., smaller ε). These observations align with the experimental results illustrated in Fig. 2 and Fig. 3. However, the improved training performance comes at the cost of increasing communication overhead, leading to longer latency per training round. Therefore, it is essential to optimize MS and client-side MA to accelerate SFL under communication and computing constraints.

V. Problem Formulation

The convergence analysis results quantify the impact of client-side MA and MS on model convergence and communication efficiency. In this section, guided by the convergence bounds in Section IV, we formulate a joint client-side MA and MS optimization problem. The objective is to minimize the training latency of AdaptSFL over edge networks with heterogeneous participating devices. Subsequently, we propose a resource-adaptive client-side MA and MS strategy in Section VI. For clarity, the decision variables and definitions are listed below.

- I: $I \in \mathbb{N}^+$ denotes the client-side MA decision variable, which indicates that the forged client-side sub-model is aggregated every I training rounds.
- $\boldsymbol{\mu}$: $\mu_{i,j} \in \{0,1\}$ is the MS decision variable, where $\mu_{i,j} = 1$ indicates that the j-th neural network layer is selected as the cut layer for edge device i, and 0 otherwise. $\boldsymbol{\mu} = [\mu_{1,1}, \mu_{1,2}, \ldots, \mu_{N,L}]$ represents the collection of MS decisions.

A. Training Latency Analysis

In this section, we provide the training latency analysis for each step of AdaptSFL. Without loss of generality, we focus on one training round for analysis. The training round number index t is omitted for notational simplicity. To begin, we analyze the latency of split training stage for one training round as follows.[3]

a1) Client-side Model Forward Propagation Latency: In this step, each edge device utilizes its local dataset to conduct the

[3]Consistent with prior studies [11], [30], [31], [32], [44], we ignore the cost of network and computing capacity probing in our analysis, as these costs are negligible compared to the excessive communication and computation costs of the training process. Moreover, lightweight tools such as iperf [45] for bandwidth measurement and Linux utilities like top for monitoring computing load are highly efficient, enabling efficient probing with minimal overhead.

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.

client-side FP. The computing workload (in FLOPs[4]) of the client-side FP for edge device i per data sample is denoted by $\Phi_{c,i}^{F}(\boldsymbol{\mu}) = \sum_{j=1}^{L} \mu_{i,j}\rho_j$, where ρ_j is the FP computing workload of propagating the first j layer of neural network for one data sample. Considering that each edge device randomly draws a mini-batch containing b data samples for executing the client-side FP, the client-side FP latency of edge device i is calculated as

$$T_i^F = \frac{b\,\Phi_{c,i}^{F}(\boldsymbol{\mu})}{f_i},\ \forall i \in \mathcal{N}, \tag{16}$$

where f_i denotes the computing capability of edge device i, i.e., the number of float-point operations per second (FLOPS).

a2) Activations Transmissions Latency: After completing the client-side FP, each edge device sends the activations generated at the cut layer to the edge server. Let $\Gamma_{a,i}(\boldsymbol{\mu}) = \sum_{j=1}^{L} \mu_{i,j}\psi_j$ represent the data size (in bits) of activations for edge device i, where ψ_j denotes the data size of activations at the cut layer j. Since each edge device employs a mini-batch with b data samples, the activations transmissions latency of edge device i is determined as

$$T_{a,i}^{U} = \frac{b\Gamma_{a,i}(\boldsymbol{\mu})}{r_i^U},\ \forall i \in \mathcal{N}. \tag{17}$$

where r_i^U is uplink transmission rate from edge device i to the edge server.

a3) Server-side Model Forward Propagation and Back-propagation Latency: This step involves executing the server-side FP and BP with collected activations from all participating edge devices. Let $\Phi_s^F(\boldsymbol{\mu}) = \sum_{i=1}^{N}\sum_{j=1}^{L} \mu_{i,j}(\rho_L - \rho_j)$ represent the computing workload of the server-side FP for each data sample. Similarly, the computing workload of the server-side BP per data sample is denoted by $\Phi_s^B(\boldsymbol{\mu}) = \sum_{i=1}^{N}\sum_{j=1}^{L} \mu_{i,j}(\varpi_L - \varpi_j)$, where ϖ_j is the BP computing workload of propagating the first j layer of neural network for one data sample. Consequently, the server-side model FP and BP latency can be obtained from

$$T_s^F = \frac{b\,\Phi_s^F(\boldsymbol{\mu})}{f_s} \tag{18}$$

and

$$T_s^B = \frac{b\,\Phi_s^B(\boldsymbol{\mu})}{f_s}, \tag{19}$$

where f_s denotes the computing capability of the edge server.

a4) Activations' Gradients Transmissions Latency: After the server-side BP is completed, the edge server sends the activations' gradients to the corresponding edge devices. Let $\Gamma_{g,i}(\boldsymbol{\mu}) = \sum_{j=1}^{L} \mu_{i,j}\chi_j$ represent the data size of activations' gradients for edge device i, where χ_j denotes the data size of activations' gradients at the cut layer j. Therefore, the activations' gradients transmissions latency of edge device i is expressed as

$$T_{g,i}^{D} = \frac{b\Gamma_{g,i}(\boldsymbol{\mu})}{r_i^D},\ \forall i \in \mathcal{N}. \tag{20}$$

where r_i^D is downlink transmission rate from edge server to edge device i.

a5) Client-side Model Back-propagation Latency: In this step, each edge device executes client-side BP according to the received activations' gradients. Let $\Phi_{c,i}^{B}(\boldsymbol{\mu}) = \sum_{j=1}^{L} \mu_{i,j}\varpi_j$ denote the computing workload of the client-side BP for edge device i per data sample. As a result, the client-side BP latency of edge device i can be obtained by

$$T_i^B = \frac{b\,\Phi_{c,i}^{B}(\boldsymbol{\mu})}{f_i},\ \forall i \in \mathcal{N}. \tag{21}$$

Then, we analyze the latency of client-side MA stage every I training round as follows.

b1) Model Uplink Transmissions Latency: In this step, each edge device transmits its client-side sub-model to the fed server, while edge server uploads the server-side non-common sub-models to the fed server for client-side MA. Let $\Lambda_{c,i}(\boldsymbol{\mu}) = \sum_{j=1}^{L} \mu_{i,j}\delta_j$ represent the data size of client-side sub-model for edge device i, where δ_j is the data size of client-side sub-model with the cut layer j. The total data size of the exchanged server-side non-common sub-models is denoted by $\Lambda_s(\boldsymbol{\mu}) = N \max_i \left\{ \sum_{j=1}^{L} \mu_{i,j}\delta_j \right\} - \sum_{i=1}^{N}\sum_{j=1}^{L} \mu_{i,j}\delta_j$. Therefore, the uplink transmissions latency for the client-side sub-model and server-side non-common sub-models are expressed as

$$T_{c,i}^{U} = \frac{\Lambda_{c,i}(\boldsymbol{\mu})}{r_{i,f}^U},\ \forall i \in \mathcal{N} \tag{22}$$

and

$$T_s^U = \frac{\Lambda_s(\boldsymbol{\mu})}{r_{s,f}}, \tag{23}$$

where $r_{i,f}^U$ and $r_{s,f}$ represent the transmissions rate from edge device i and edge server to the fed server, respectively.

b2) Client-side Model Aggregation: The fed server aggregates the client-side sub-models and server-side non-common sub-models to forge the aggregated client-specific model. For simplicity, the latency for this part is ignored since the aggregation delay is typically much smaller than other steps [48], [49].

b3) Model Downlink Transmissions Latency: After completing client-side MA, the fed server sends the updated client-side sub-model and server-side non-common sub-model to the corresponding edge devices and edge server. Similarly, the downlink transmission latency for the client-side sub-model and server-side non-common sub-models are calculated by

$$T_{c,i}^{D} = \frac{\Lambda_{c,i}(\boldsymbol{\mu})}{r_{i,f}^D},\ \forall i \in \mathcal{N} \tag{24}$$

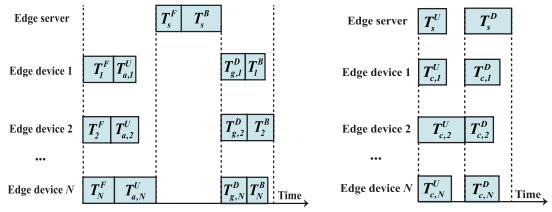
and

$$T_s^D = \frac{\Lambda_s(\boldsymbol{\mu})}{r_{f,s}}, \tag{25}$$

where $r_{i,f}^D$ and $r_{f,s}$ denote the transmission rate from the fed server to edge device i and edge server, respectively.

[4]Consistent with prior distributed machine learning literature [30], [32], [46], [47], we adopt FLOPs as a standardized, hardware-independent metric to quantify computing cost, thus enabling fair comparisons across different model architectures and simulation settings.

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.



(a) Split training. (b) Client-side MA.

Fig. 4. An illustration of split training and client-side MA stages.

B. Joint Client-Side Model Aggregation and Model Splitting Problem Formulation

This section presents the problem formulation of client-side MA and MS with the objective of minimizing the time consumed for model convergence over edge networks. As illustrated in Fig. 4, considering that split training is executed per training round and client-side MA occurs every I training round, the total training latency for I training rounds can be derived as

$$\begin{aligned} &T(I,\boldsymbol{\mu}) \\ &= I\left\{\max_i\left\{T_i^F+T_{a,i}^U\right\}+T_s^F+T_s^B+\max_i\left\{T_{g,i}^D+T_i^B\right\}\right\} \\ &\quad+\max_i\left\{T_{c,i}^U,T_s^U\right\}+\max_i\left\{T_{c,i}^D,T_s^D\right\}. \end{aligned} \tag{26}$$

Recalling the derived convergence bound in (13), client-side MA interval balances the tradeoffs between communication overhead and training convergence, while MS significantly impacts communication-computing overhead, as well as model convergence. Hence, jointly optimizing the client-side MA interval and MS is imperative to expedite the training process. Observing these facts, we formulate the following optimization problem to minimize the time consumed for model convergence:

$$\begin{aligned} \mathcal{P}:\ &\min_{I,\boldsymbol{\mu}} MT(I,\boldsymbol{\mu}) \\ &\text{s.t. C1}: \frac{1}{R}\sum_{t=1}^{R}\mathbb{E}[\|\nabla_{\mathbf{w}}f(\mathbf{w}^{t-1})\|^2]\le\varepsilon, \\ &\quad\text{C2}: \mu_{i,j}\in\{0,1\},\quad \forall i\in\mathcal{N},\ j=1,2,\ldots,L, \\ &\quad\text{C3}: \sum_{j=1}^{L}\mu_{i,j}=1,\quad \forall i\in\mathcal{N}, \\ &\quad\text{C4}: I\in\mathbb{N}^+. \end{aligned} \tag{27}$$

where $M=\frac{R}{I}$ denotes the number of rounds of model aggregation for achieving model convergence. Constrain C1 guarantees global convergence accuracy; C2 and C3 ensure that each edge device has a single cut layer, so that the global model is partitioned into client-side and server-side sub-models; C4 denotes that the client-side MA decision variable is a positive integer.

Problem (27) is a combinatorial problem with a non-convex mixed-integer non-linear objective function, which is NP-hard in general. In this case, achieving an optimal solution via polynomial-time algorithms is infeasible.

VI. Solution Approach

In this section, we present an efficient iterative algorithm by decoupling the problem (27) into MA and MS subproblems. For each subproblem, the optimal solution can be obtained. We first utilize Corollary 1 to explicitly express M. It is clear that M is proportional to the objective function, and thus the objective function is minimized if and only if inequality (15) holds as an equality. Thus, the original problem (27) can be converted into

$$\begin{aligned} \mathcal{P}':\ &\min_{I,\boldsymbol{\mu}}\Theta(I,\boldsymbol{\mu}) \\ &\text{s.t. C2}-\text{C4}, \end{aligned} \tag{28}$$

where

$$\Theta(I,\boldsymbol{\mu})=\frac{2\vartheta}{\gamma I\left(\varepsilon-\frac{\beta\gamma\sum_{j=1}^{L}\sigma_j^2}{N}-4\beta^2\gamma^2I^2\sum_{j=1}^{L_c}G_j^2\right)}T(I,\boldsymbol{\mu}).$$

Problem (28) is still a mixed-integer non-linear programming, and the non-convexity of the objective function makes it still intractable. To solve this problem, we introduce a set of constants $\widetilde{\mathbf{G}}=\left[\widetilde{G}_1^2,\widetilde{G}_2^2,\ldots,\widetilde{G}_L^2\right]$, where $\widetilde{G}_j^2$ denotes the cumulative sum of the bounded second order moments for the first j layers of neural network, defined as

$$\widetilde{G}_j^2=\sum_{k=1}^{j}G_k^2. \tag{29}$$

Therefore, $\sum_{j=1}^{L_c}G_j^2$ in the objective function can be reformulated as $\max_i\left\{\sum_{j=1}^{L}\mu_{i,j}\widetilde{G}_j^2\right\}$. Additionally, we introduce a set of auxiliary variables $\mathbf{T}=[T_1,T_2,\ldots,T_6]$ to linearize the objective function, i.e., $\max_i\left\{\sum_{j=1}^{L}\mu_{i,j}\widetilde{G}_j^2\right\}\le T_1$, $\max_i\left\{\sum_{j=1}^{L}\mu_{i,j}\delta_j\right\}\le T_2$, $\max_i\left\{T_i^F+T_{a,i}^U\right\}\le T_3$, $\max_i\left\{T_{g,i}^D+T_i^B\right\}\le T_4$, $\max_i\left\{T_{c,i}^U,T_s^U\right\}\le T_5$, and $\max_i\left\{T_{c,i}^D,T_s^D\right\}\le T_6$. Problem (28) can therefore be transformed into

$$\begin{aligned} \mathcal{P}'':\ &\min_{I,\boldsymbol{\mu},\mathbf{T},}\Theta'(I,\boldsymbol{\mu},\mathbf{T}) \\ &\text{s.t. C2}-\text{C4}, \\ &\quad\text{R1}: \sum_{j=1}^{L}\mu_{i,j}\widetilde{G}_j^2\le T_1,\quad\forall i\in\mathcal{N}, \\ &\quad\text{R2}: \sum_{j=1}^{L}\mu_{i,j}\delta_j\le T_2,\quad\forall i\in\mathcal{N}, \\ &\quad\text{R3}: \frac{b\sum_{j=1}^{L}\mu_{i,j}\rho_j}{f_i}+\frac{b\sum_{j=1}^{L}\mu_{i,j}\psi_j}{r_i^U}\le T_3,\quad\forall i\in\mathcal{N}, \\ &\quad\text{R4}: \frac{b\sum_{j=1}^{L}\mu_{i,j}\chi_j}{r_i^D}+\frac{b\sum_{j=1}^{L}\mu_{i,j}\varpi_j}{f_i}\le T_4,\quad\forall i\in\mathcal{N}, \\ &\quad\text{R5}: \frac{\sum_{j=1}^{L}\mu_{i,j}\delta_j}{r_{i,f}^U}\le T_5,\quad\forall i\in\mathcal{N}, \end{aligned}$$

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.

$$\text{R6}: \frac{NT_2 - \sum_{i=1}^{N}\sum_{j=1}^{L}\mu_{i,j}\delta_j}{r_{s,f}} \leq T_5,$$
$$\text{R7}: \frac{\sum_{j=1}^{L}\mu_{i,j}\delta_j}{r_{i,f}^{D}} \leq T_6 \quad \forall i \in \mathcal{N},$$
$$\text{R8}: \frac{NT_2 - \sum_{i=1}^{N}\sum_{j=1}^{L}\mu_{i,j}\delta_j}{r_{f,s}} \leq T_6, \tag{30}$$

where

$$\Theta'(I,\boldsymbol{\mu},\mathbf{T}) = \frac{2\vartheta\left\{I\left\{T_3+T_s^F+T_s^B+T_4\right\}+T_5+T_6\right\}}{\gamma I\left(\varepsilon - \frac{\beta\gamma\sum_{j=1}^{L}\sigma_j^2}{N} - 4\beta^2\gamma^2 I^2 T_1\right)}. \tag{31}$$

From problem (30), we observe that the introduced auxiliary variables are tightly coupled with the original decision variables, which makes it difficult to solve the problem directly. Hence, we decompose the original problem (30) into two less complicated subproblems based on different decision variables and develop efficient algorithms to tackle each subproblem.

We fix the variables $\boldsymbol{\mu}$ and $\mathbf{T}$ to investigate the subproblem involving client-side MA interval, which is expressed as

$$\mathcal{P}_1: \min_{I} \Theta'(I) \quad \text{s.t. C4.} \tag{32}$$

Then, we have the following theorem:

Theorem 2: The optimal client-side MA interval is given by

$$I^* = \begin{cases} 1 & I' \leq 1 \\ \arg\min_{I\in\{\lfloor I'\rfloor,\lceil I'\rceil\}}\Theta'(I) & I' > 1, \end{cases} \tag{33}$$

where I' with $\Xi(I') = 0$ can be easily obtained by Newton-Raphson method and $\Xi(I)$ is defined in (47).

Proof: See Appendix C. □

Therefore, we obtain an optimal solution for problem (32).

By fixing the decision variable I, we convert problem (30) into a standard mixed-integer linear fractional programming (MILFP) with respect to $\boldsymbol{\mu}$ and $\mathbf{T}$, which is given by

$$\mathcal{P}_2: \min_{\boldsymbol{\mu},\mathbf{T}} \Theta'(\boldsymbol{\mu},\mathbf{T}) \quad \text{s.t. C2, C3, R1} - \text{R8.} \tag{34}$$

We utilize the Dinkelbach algorithm [50] to efficiently solve problem (34) by reformulating it as mixed-integer linear programming, which achieves the optimal solution [51], [52]. Introducing the fractional parameter λ, we reformulate problem (34) as

$$\mathcal{P}_2': \min_{\boldsymbol{\mu},\mathbf{T}} \Upsilon(\lambda,\boldsymbol{\mu},\mathbf{T}) = Q(\boldsymbol{\mu},\mathbf{T}) - \lambda P(\boldsymbol{\mu},\mathbf{T}) \quad \text{s.t. C2, C3, R1} - \text{R8}, \tag{35}$$

where

$$Q(\boldsymbol{\mu},\mathbf{T}) = 2\vartheta\left\{I\left\{T_3+T_s^F+T_s^B+T_4\right\}+T_5+T_6\right\},$$
$$P(\boldsymbol{\mu},\mathbf{T}) = \gamma I\left(\varepsilon - \frac{\beta\gamma\sum_{j=1}^{L}\sigma_j^2}{N} - 4\beta^2\gamma^2 I^2 T_1\right).$$

Algorithm 2 Dinkelbach Method for Solving Problem (34)

Input: $\lambda^{(0)}$ satisfying $\Upsilon(\lambda^{(0)},\boldsymbol{\mu}^{(0)},\mathbf{T}^{(0)}) \geq 0$, $\boldsymbol{\mu}^{(0)}$, $\mathbf{T}^{(0)}$ and ε_d.
Output: $\boldsymbol{\mu}^*$ and λ^*.
1: Initialization: $n \leftarrow 1$.
2: **repeat**
3: Solve problem (35) with $\lambda = \lambda^{(n)}$ to obtain $\boldsymbol{\mu}^{(n)}$ and $\mathbf{T}^{(n)}$
4: Calculate $\Upsilon(\lambda^{(0)},\boldsymbol{\mu}^{(n)},\mathbf{T}^{(n)})$
5: $\lambda^{(n+1)} \leftarrow \frac{Q(\boldsymbol{\mu}^{(n)},\mathbf{T}^{(n)})}{P(\boldsymbol{\mu}^{(n)},\mathbf{T}^{(n)})}$,
6: $n \leftarrow n+1$
7: **until** $|\Upsilon(\lambda^{(n)},\boldsymbol{\mu}^{(n)},\mathbf{T}^{(n)})| \leq \varepsilon_d$

The Dinkelbach algorithm iteratively solves problem (35), which is described as follows: We first initialize the fractional parameter $\lambda^{(0)}$ such that $\Upsilon(\lambda^{(0)},\boldsymbol{\mu}^{(0)},\mathbf{T}^{(0)}) \geq 0$. At each iteration, we obtain the optimal solution by solving problem (35) with $\lambda^{(n)}$. After that, we update the fractional parameter $\lambda^{(n+1)}$. We repeat the iterations until $|\Upsilon(\lambda^{(n)},\boldsymbol{\mu}^{(n)},\mathbf{T}^{(n)})| < \varepsilon_d$, where ε_d is the convergence tolerance. The pseudo-code of the Dinkelbach algorithm for solving problem (34) is given in Algorithm 2.

Algorithm 3 BCD-Based Algorithm

Input: $I^{(0)}$, $\boldsymbol{\mu}^{(0)}$, $\mathbf{T}^{(0)}$ and ε_b.
Output: I^* and $\boldsymbol{\mu}^*$
1: Initialization: $\tau \leftarrow 0$.
2: **repeat**
3: $\tau \leftarrow \tau + 1$
4: Update $I^{(\tau)}$ based on Theorem 2
5: Update $\boldsymbol{\mu}^{(\tau)}$ and $\mathbf{T}^{(\tau)}$ based on Algorithm 2
6: **until** $|\Theta'(I^{(\tau)},\boldsymbol{\mu}^{(\tau)},\mathbf{T}^{(\tau)}) - \Theta'(I^{(\tau-1)},\boldsymbol{\mu}^{(\tau-1)},\mathbf{T}^{(\tau-1)}| \leq \varepsilon_b$

As aforementioned, we decompose the original problem (27) into two tractable subproblems $\mathcal{P}1$ and $\mathcal{P}2$ according to different decision variables and develop efficient algorithms to solve each subproblem optimally. Based on this, we propose an iterative block-coordinate descent (BCD)-based algorithm [53] to solve problem (27), which is shown in Algorithm 3. The key parameters required for executing algorithm (e.g., β, G_j^2 and σ_j^2) are estimated following the approach in [25] and lightweight tools can be utilized to measure network resources, such as iperf [45] for bandwidth estimation and system monitoring tools (e.g., Linux top or Android performance APIs) for computing speed. It is noted that, given dynamic network and training conditions, Algorithm 3 can be conducted every I rounds or longer to accelerate SFL under time-varying edge environments, as illustrated in Algorithm 1.

VII. Performance Evaluation

This section provides numerical results to evaluate the learning performance of the proposed AdaptSFL framework and the effectiveness of tailored client-side MA and MS strategy.

A. Simulation Setup

We implement AdaptSFL using Python 3.7 and PyTorch 1.9.1., and train it on a ThinkPad P17 Gen1 laptop equipped

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SIMULATION PARAMETERS

Parameter	Value	Parameter	Value
f_s	20 TFLOPS	f_i	$[1, 2]$ TFLOPS
N	20	$r_i^U/r_{i,f}^U$	$[75, 80]$ Mbps
$r_i^D/r_{i,f}^D$	370 Mbps	r_s^U/r_s^D	400 Mbps
b	16	γ	5×10^{-4}

with an NVIDIA Quadro RTX 3000 GPU, Intel i9-10885H CPUs and 4TB SSD. In our experiments, we deploy N edge devices and N is set to 20 by default unless specified otherwise. The computing capability of each edge device is uniformly distributed within $[1, 2]$ TFLOPS, and the computing capability of the server is set to 20 TFLOPS. The uplink transmission rates from edge device i to the edge server and fed server follow uniform distribution wit hin $[75, 80]$ Mbps, and the corresponding downlink rates are set to 370 Mbps. It is noted that the computing capability of each edge device and its uplink transmission rates to both edge server and fed server vary across every training round, reflecting dynamic variation in network resources throughout the training process. For convenience, the inter-server transmission rate, namely r_s^U and r_s^D, are identically set to 400 Mbps. The mini-batch size and learning rate are set to 16 and 5×10^{-4}. For readers' convenience, the detailed simulation parameters are summarized in Table II.

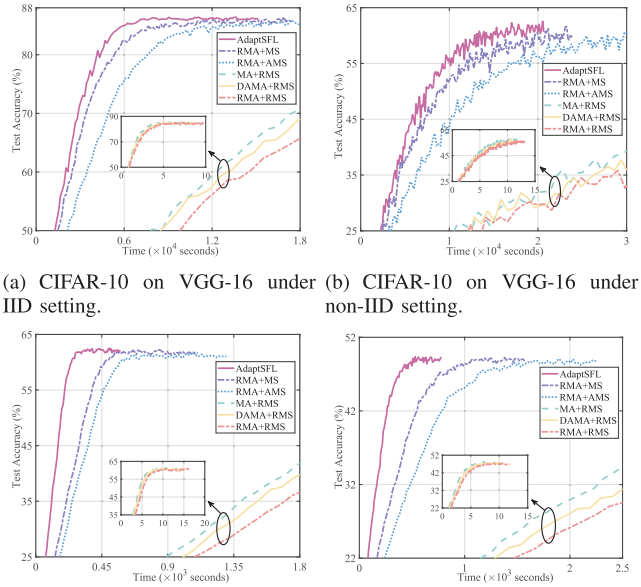
We adopt the object detection dataset CIFAR-10 and CIFAR-100 [54] to evaluate the learning performance of AdaptSFL. The CIFAR-10 dataset contains 10 distinct categories of object images, such as airplanes, automobiles and trucks, and comprises 50000 training samples as well as 10000 test samples. The CIFAR-100 dataset includes object images from 100 categories, with each category containing 500 training samples and 100 test samples. Furthermore, we conduct experiments under IID and non-IID data settings. The data samples are shuffled and distributed evenly to all edge devices in the IID setting. In the non-IID setting [14], [44], [55], we sort the data samples by labels, divide them into 40 shards, and distribute 2 shards to each of the 20 edge devices.

We deploy the well-known VGG-16 [56] and ResNet-18 [57] neural networks. VGG-16 is a classical deep convolutional neural network comprised of 13 convolution layers and 3 fully connected layers, while ResNet-18 is a residual neural network composed of 17 convolutional layers and 1 fully connected layer.

B. Performance Evaluation of AdaptSFL Framework

In this section, we assess the overall performance of AdaptSFL framework in terms of test accuracy and convergence speed. In addition, we evaluate the robustness of AdaptSFL to varying network resources. To investigate the advantages of AdaptSFL framework, we compare it with five benchmarks:

- **RMA+MS:** The RMA+MS benchmark deploys a random client-side MA strategy (i.e., the client-side MA interval I is randomly drawn from 1 to 25 during model training.), and employs the adaptive MS scheme in Section VI.
- **MA+RMS:** The MA+RMS benchmark utilizes the adaptive client-side MA strategy in Section VI and adopts a random MS scheme (i.e., randomly selecting model split points during model training).



(a) CIFAR-10 on VGG-16 under IID setting. (b) CIFAR-10 on VGG-16 under non-IID setting.

(c) CIFAR-100 on ResNet-18 under IID setting. (d) CIFAR-100 on ResNet-18 under non-IID setting.

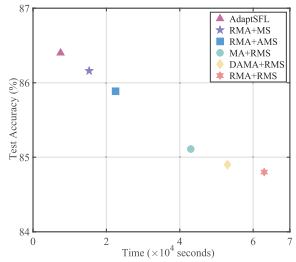
Fig. 5. The training performance for CIFAR-10 and CIFAR-100 datasets under IID and non-IID settings using VGG-16 and ResNet-18.

- **RMA+RMS:** The RMA+RMS benchmark employs the random client-side MA and MS strategy.
- **DAMA+RMS:** The DAMA+RMS benchmark employs the depth-aware adaptive client-side MA scheme [23] and utilizes the RMS strategy.
- **RMA+AMS:** The RMA+HMS benchmark employs the RMA scheme and utilizes a adaptive resource-heterogeneity-aware MS strategy [58].

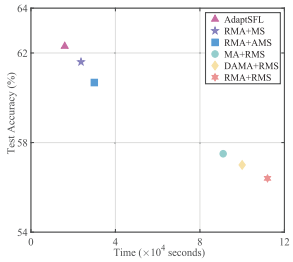
Fig. 5 illustrates the training performance of AdaptSFL and five benchmarks on the CIFAR-10 and CIFAR-100 datasets. AdaptSFL exhibits faster convergence speed and higher test accuracy compared to the other five benchmarks as the model converges. Notably, AdaptSFL, RMA+MS, and RMA+AMS converge significantly faster than MA+RMS, DAMA+RMS, and RMA+RMS owing to adaptive MS, which strikes a good balance between communication-computing overhead and training convergence speed. Furthermore, the comparison between AdaptSFL and RMA+MS and RMA+AMS, shows that the tailored client-side MA strategy can further accelerate model training without deteriorating training accuracy. The performance gap between RMA+MS and RMA+AMS, as well as MA+RMS and DAMA+RMS, is mainly because RMA+AMS and DAMA+RMS are heuristic and have not been systematically optimized based on the interplay between model convergence and client-side model aggregation and model splitting. By comparing Fig. 5(a) with Fig. 5(b), and Fig. 5(c) with Fig. 5(d), reveals that the convergence speed of AdaptSFL and other five benchmarks are slower under non-IID setting than under IID setting.

Fig. 6 presents converged accuracy and time (i.e., the incremental increase in test accuracy lower than 0.02% for 5 consecutive training rounds) for CIFAR-10 and CIFAR-100 datasets. Comparing AdaptSFL with RMA+MS and MA+RMS reveals a substantial impact of client-side MA interval and MS on converged accuracy and time. Moreover, the impact of MS on model training outweighs that of MA interval. This is because the choice of model split points directly determines the overall aggregation interval of

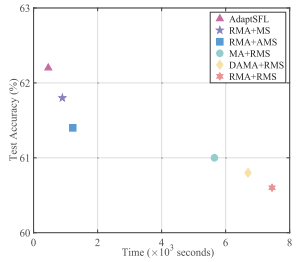
Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.



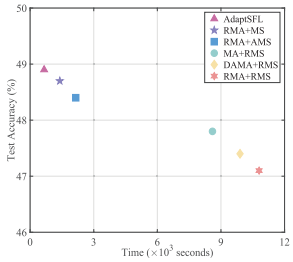
(a) CIFAR-10 on VGG-16 under IID setting.



(b) CIFAR-10 on VGG-16 under non-IID setting.

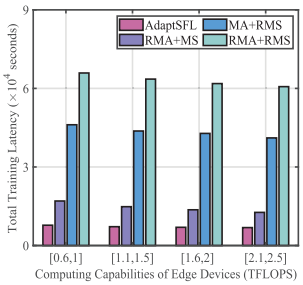


(c) CIFAR-100 on ResNet-18 under IID setting.

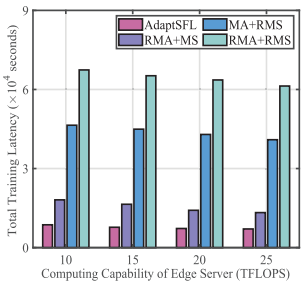


(d) CIFAR-100 on ResNet-18 under non-IID setting.

Fig. 6. The converged accuracy and time for CIFAR-10 and CIFAR-100 datasets under IID and non-IID settings using VGG-16 and ResNet-18.



(a) Computing capabilities of edge devices.

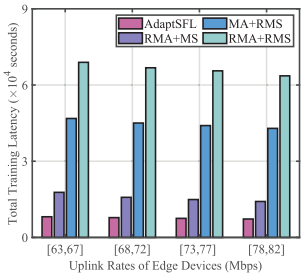


(b) Computing capabilities of edge server.

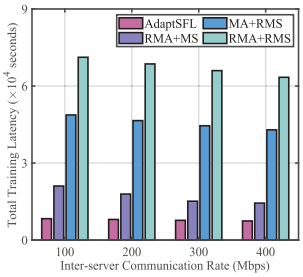
Fig. 7. The converged time versus network computing resources on CIFAR-10 under IID setting using VGG-16.

the global model, e.g., shallower model split points (fewer client-side sub-model layers) imply larger portions of the model are aggregated at each training round. In the IID setting, unoptimized MS leads to 1.5% accuracy degradation and nearly five-fold convergence deceleration on the CIFAR-10 dataset, which is 6.2 and 4.6 times that of unoptimized MA. Furthermore, this difference is more pronounced under the non-IID setting, reaching approximately 7.3 and 10 times, due to model bias caused by discrepancies across local datasets, consistent with FL. The comparison between AdaptSFL and RMA+RMS shows that AdaptSFL achieves a convergence speed improvement of at least 7.7 times over its counterpart without optimization while guaranteeing training accuracy, demonstrating the superior performance of the AdaptSFL framework. RMA+AMS and DAMA+RMS exhibit slower model convergence and lower accuracy compared to RMA+MS and MA+RMS, primarily because they are not designed based on model convergence.

Fig. 7–8 show the converged time versus network computing and communication resources on CIFAR-10 under IID setting. AdaptSFL exhibits significantly shorter converged times than the other five benchmarks across varying computing and communication resources. As network resources decline, the

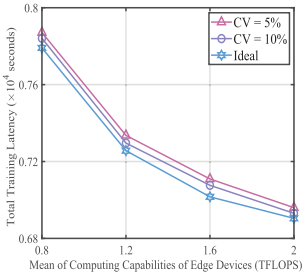


(a) Uplink rates of edge devices.

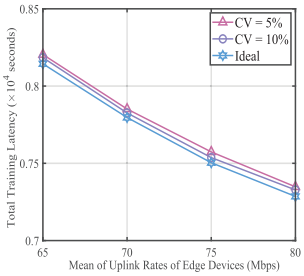


(b) Inter-server communication rate.

Fig. 8. The converged time versus network communication resources on CIFAR-10 under IID setting using VGG-16.



(a) Computing capability uncertainty.



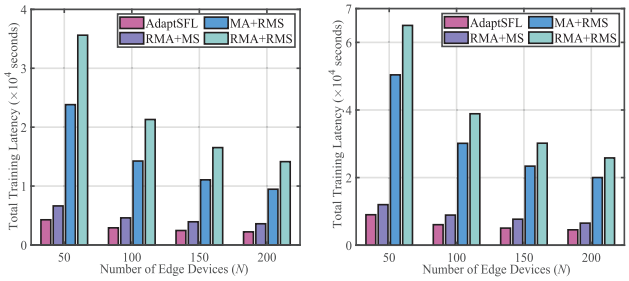
(b) Uplink transmission rate uncertainty.

Fig. 9. The impact of network resource measurement uncertainty on converged time on CIFAR-10 under IID setting using VGG-16.

convergence speed of RMA+RMS noticeably slows down. This is because it neither optimizes model split points to strike a balance between computing and communication overheads nor optimizes the client-side MA interval for expediting model training. The comparison between RMA+MS and MA+RMS reveals that optimizing MA interval or MS somewhat mitigates the rise in convergence time with reduced network resources. In contrast, the converged time of AdaptSFL only experiences a slight increase with diminishing network resources, owing to its resource-adaptive client-side MA and MS design. Specifically, as network resources vary, AdaptSFL can optimize MS according to resource conditions and accelerate model convergence by adjusting client-side MA interval to achieve the minimum rounds of model aggregation required for model convergence. This demonstrates the robustness of AdaptSFL to network resources and highlights the adaptability of client-side MA and MS strategies to changes in network resources. However, in real-world scenarios, computing and communication resources fluctuate during training, leading to discrepancies between measured and actual network resource conditions. To investigate the impact of measurement error, we introduce Gaussian noise with diverse coefficient of variation (CV) to model the fluctuation in data rates and computing capabilities [59], [60]. Fig. 9 illustrates that AdaptSFL remains robust across different CVs, causing only minor variations in converged time. This validates the effectiveness of our adaptive MA and MS strategies in dynamic resource fluctuations.

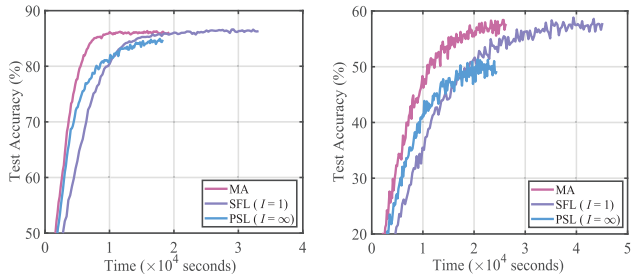
Fig. 10 presents the converged time versus the number of edge devices on CIFAR-10 under IID and non-IID settings. It is seen that AdaptSFL consistently achieves significantly lower training latency compared to MA+RMS, RMA+MS, and RMA+RMS across all configurations of N. Notably, as the number of edge devices grows, the performance gap between AdaptSFL and all baselines becomes increasingly pronounced, underscoring the scalability of AdaptSFL in handling

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.



(a) CIFAR-10 under IID setting. (b) CIFAR-10 under non-IID setting.

Fig. 10. The converged time versus number of edge devices on CIFAR-10 under IID and non-IID setting using VGG-16.



(a) CIFAR-10 under IID setting. (b) CIFAR-10 under non-IID setting.

Fig. 11. Ablation experiments for MA strategy on the CIFAR-10 dataset under IID and non-IID setting using VGG-16 with $L_c = 8$, where parallel split learning (PSL) [19] is a special case of SFL when $I = \infty$.

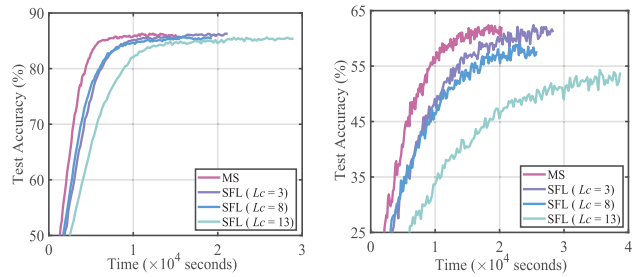
large-scale edge networks. Additionally, the total training latency of the non-IID setting is higher than that of the IID setting. This behavior reflects the inherent complexity of training under non-IID data distributions, which often leads to slower convergence due to increased heterogeneity among client-side models. Nonetheless, AdaptSFL maintains its advantage over the baseline methods, demonstrating its robustness and adaptability in tackling diverse and non-uniform data distributions.

C. Ablation Study of The AdaptSFL Framework

In this section, we conduct ablation experiments to illustrate the effectiveness of each component in AdaptSFL.

Fig. 11 shows the impact of MA interval on training performance for the CIFAR-10 dataset. PSL [19] exhibits the slowest convergence rate and lowest convergence accuracy due to its lack of client-side MA, leading to inferior generalization performance of local client-side sub-models on the global dataset. In contrast, the proposed resource-adaptive MA strategy expedites model convergence while retaining accuracy comparable to SFL with $I = 1$ (equivalent to centralized learning performance). The proposed client-side MA strategy achieves 1.5% and 5.1% higher convergence accuracy than PSL, and the convergence time is improved by a factor of 1.8 and 1.9 compared to SFL with $I = 1$ under IID and non-IID settings. This improvement stems from its ability to dynamically adjust MA interval to achieve the minimum communication-computing latency required for model convergence. Hence, the effectiveness of the proposed MA scheme is demonstrated.

Fig. 12 presents the impact of MS on training performance for the CIFAR-10 dataset. It can be seen that the convergence accuracy and speed decrease with the deepening of model split point L_c, consistent with the derived convergence boundary in Eqn. (13). This is because deeper model split points result in the smaller portion of global model being aggregated per



(a) CIFAR-10 under IID setting. (b) CIFAR-10 under non-IID setting.

Fig. 12. Ablation experiments for MS scheme on the CIFAR-10 dataset under IID and non-IID setting using VGG-16 with $I = 15$.

training round, leading to a lower overall update frequency of the whole model. Furthermore, the impact of MS on convergence accuracy and speed is more significant under non-IID than IID settings. This is attributed to the higher sensitivity of local dataset distribution discrepancies to MS under non-IID setting. The comparisons between MS and the other five benchmarks reveal that the proposed MS strategy can accelerate model convergence while guaranteeing training performance. This demonstrates the superior performance of the tailored MS strategy.

VIII. Conclusion

In this paper, we proposed a split federated learning (SFL) framework, named AdaptSFL, to minimize the training latency of SFL over edge networks. We first derived a convergence bound of SFL in terms of model aggregation intervals and model split points. Then, we devised a resource-adaptive client-side model aggregation (MA) scheme and a model splitting (MS) scheme to minimize end-to-end training latency subject to a training loss constraint under resource and data heterogeneity. Simulation results demonstrated that our proposed AdaptSFL framework achieves the target accuracy with significantly less time budget compared to benchmarks.

While this work has demonstrated the potential of integrating SFL and edge computing paradigm, more research efforts could be made to further enhance the efficiency of SFL. The optimization of other key training hyperparameters, such as batch size and learning rate, is worth further exploration. Moreover, future work could also incorporate network and computing probing costs into the optimization framework. This is particularly pertinent in highly dynamic network environments, where frequent state changes might exacerbate the relative impact of probing overhead. In addition, further research is needed to prototype SFL in real-world edge computing systems to further validate training performance under practical deployment constraints. All these considerations could further enhance the efficiency of SFL under resource-constrained systems.

Appendix

A. Proof of Lemma 1

Fix training round $t \geq 1$. Considering the largest $t_0 \leq t$ that satisfies $t_0 \bmod I = 0$ (Note that such t_0 must exist and $t - t_0 \leq I$.) Recalling the Eqn. (5), Eqn. (6) and Eqn. (8) for updating the model weights, that

$$\mathbf{h}_{c,i}^{t} = \mathbf{h}_{c}^{t_0} - \gamma \sum_{\tau=t_0+1}^{t} \mathbf{g}_{c,i}^{\tau} \tag{36}$$

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.

By (7), we have

$$\mathbf{h}_c^t = \mathbf{h}_c^{t_0} - \gamma \sum_{\tau=t_0+1}^{t} \frac{1}{N} \sum_{i=1}^{N} \mathbf{g}_{c,i}^{\tau}$$

Thus, we have

$$\begin{aligned}
&\mathbb{E}[\|\mathbf{h}_c^t - \mathbf{h}_{c,i}^t\|^2] \\
&= 1_{\{I>1\}} \mathbb{E}\left[\|\gamma \sum_{\tau=t_0+1}^{t} \frac{1}{N} \sum_{i=1}^{N} \mathbf{g}_{c,i}^{\tau} - \gamma \sum_{\tau=t_0+1}^{t} \mathbf{g}_{c,i}^{\tau}\|^2\right] \\
&= 1_{\{I>1\}} \gamma^2 \mathbb{E}\left[\| \sum_{\tau=t_0+1}^{t} \frac{1}{N} \sum_{i=1}^{N} \mathbf{g}_{c,i}^{\tau} - \sum_{\tau=t_0+1}^{t} \mathbf{g}_{c,i}^{\tau}\|^2\right] \\
&\overset{(a)}{\leq} 1_{\{I>1\}} 2\gamma^2 \mathbb{E}\left[\| \sum_{\tau=t_0+1}^{t} \frac{1}{N} \sum_{i=1}^{N} \mathbf{g}_{c,i}^{\tau}\|^2 + \| \sum_{\tau=t_0+1}^{t} \mathbf{g}_{c,i}^{\tau}\|^2\right] \\
&\overset{(b)}{\leq} 1_{\{I>1\}} 2\gamma^2 (t-t_0) \mathbb{E}\left[\sum_{\tau=t_0+1}^{t} \| \frac{1}{N} \sum_{i=1}^{N} \mathbf{g}_{c,i}^{\tau}\|^2 + \sum_{\tau=t_0+1}^{t} \|\mathbf{g}_{c,i}^{\tau}\|^2\right] \\
&\overset{(c)}{\leq} 1_{\{I>1\}} 2\gamma^2 (t-t_0) \mathbb{E}\left[\sum_{\tau=t_0+1}^{t} \left(\frac{1}{N} \sum_{i=1}^{N} \|\mathbf{g}_{c,i}^{\tau}\|^2\right) + \sum_{\tau=t_0+1}^{t} \|\mathbf{g}_{c,i}^{\tau}\|^2\right] \\
&\overset{(d)}{\leq} 1_{\{I>1\}} 4\gamma^2 I^2 \sum_{j=1}^{L_c} G_j^2
\end{aligned}$$

where (a)-(c) follows by using the inequality $\|\sum_{i=1}^{n} \mathbf{z}_i\|^2 \leq n \sum_{i=1}^{n} \|\mathbf{z}_i\|^2$ for any vectors $\mathbf{z}_i$ and any positive integer n (using $n = 2$ in (a), $n = t - t_0$ in (b), and $n = N$ in (c)); and (d) follows from Assumption 2.

B. Proof of Theorem 1

For training round $t \geq 1$. By the smoothness of loss function $f(\cdot)$, we have

$$\begin{aligned}
\mathbb{E}[f(\mathbf{w}^t)] \leq& \mathbb{E}[f(\mathbf{w}^{t-1})] + \mathbb{E}[\langle \nabla_{\mathbf{w}} f(\mathbf{w}^{t-1}), \mathbf{w}^t - \mathbf{w}^{t-1} \rangle] \\
&+ \frac{\beta}{2} \mathbb{E}[\|\mathbf{w}^t - \mathbf{w}^{t-1}\|^2].
\end{aligned} \tag{37}$$

Note that

$$\begin{aligned}
&\mathbb{E}[\|\mathbf{w}^t - \mathbf{w}^{t-1}\|^2] \\
&= \mathbb{E}[\|[\mathbf{h}_c^t; \mathbf{h}_s^t] - [\mathbf{h}_c^{t-1}; \mathbf{h}_s^{t-1}]\|^2] \\
&= \mathbb{E}[\|[\mathbf{h}_c^t - \mathbf{h}_c^{t-1}; \mathbf{h}_s^t - \mathbf{h}_s^{t-1}]\|^2] \\
&= \mathbb{E}[\|\mathbf{h}_c^t - \mathbf{h}_c^{t-1}\|^2] + \mathbb{E}[\|\mathbf{h}_s^t - \mathbf{h}_s^{t-1}\|^2],
\end{aligned} \tag{38}$$

where $\mathbb{E}[\|\mathbf{h}_c^t - \mathbf{h}_c^{t-1}\|^2]$ can be bounded as

$$\begin{aligned}
\mathbb{E}[\|\mathbf{h}_c^t - \mathbf{h}_c^{t-1}\|^2] &\overset{(a)}{=} \gamma^2 \mathbb{E}\left[\|\frac{1}{N} \sum_{i=1}^{N} \mathbf{g}_{c,i}^t\|^2\right] \\
&\overset{(b)}{=} \gamma^2 \mathbb{E}\left[\|\frac{1}{N} \sum_{i=1}^{N} \left(\mathbf{g}_{c,i}^t - \nabla_{\mathbf{h}_c} f_i\left(\mathbf{h}_{c,i}^{t-1}\right)\right)\|^2\right] \\
&\quad + \gamma^2 \mathbb{E}\left[\|\frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{h}_c} f_i\left(\mathbf{h}_{c,i}^{t-1}\right)\|^2\right] \\
&\overset{(c)}{=} \frac{\gamma^2}{N^2} \sum_{i=1}^{N} \mathbb{E}[\|\mathbf{g}_{c,i}^t - \nabla_{\mathbf{h}_c} f_i\left(\mathbf{h}_{c,i}^{t-1}\right)\|^2] \\
&\quad + \gamma^2 \mathbb{E}\left[\|\frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{h}_c} f_i\left(\mathbf{h}_{c,i}^{t-1}\right)\|^2\right] \\
&\overset{(d)}{\leq} \frac{\gamma^2 \sum_{j=1}^{L_c} \sigma_j^2}{N} + \gamma^2 \mathbb{E}\left[\|\frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{h}_c} f_i\left(\mathbf{h}_{c,i}^{t-1}\right)\|^2\right].
\end{aligned} \tag{39}$$

where (a) follows from Eqn. (7) and Eqn. (36); (b) follows by observing that $\mathbb{E}[\mathbf{g}_{c,i}^t] = \nabla_{\mathbf{h}_c} f_i(\mathbf{h}_{c,i}^{t-1})$ and applying the equality $\mathbb{E}[\|\mathbf{z}\|^2] = \mathbb{E}[\|\mathbf{z} - \mathbb{E}[\mathbf{z}]\|^2] + \|\mathbb{E}[\mathbf{z}]\|^2$ that holds for any random vector $\mathbf{z}$; (c) follows because each $\mathbf{g}_{c,i}^t - \nabla_{\mathbf{h}_c} f_i(\mathbf{h}_{c,i}^{t-1})$ has zero mean and is independent across edge devices; and (d) follows from Assumption 2.

Similarly, $\mathbb{E}[\|\mathbf{h}_s^t - \mathbf{h}_s^{t-1}]\|^2$ has an upper bound:

$$\begin{aligned}
\mathbb{E}[\|\mathbf{h}_s^t - \mathbf{h}_s^{t-1}\|^2] \leq& \frac{\gamma^2 \sum_{j=L_c+1}^{L} \sigma_j^2}{N} \\
&+ \gamma^2 \mathbb{E}\left[\|\frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{h}_s} f_i\left(\mathbf{h}_{s,i}^{t-1}\right)\|^2\right].
\end{aligned} \tag{40}$$

Substituting Eqn. (39) and Eqn. (40) into Eqn. (38) yields

$$\begin{aligned}
&\mathbb{E}[\|\mathbf{w}^t - \mathbf{w}^{t-1}\|^2] \\
&\leq \frac{\gamma^2 \sum_{j=1}^{L} \sigma_j^2}{N} + \gamma^2 \mathbb{E}\left[\|\frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{h}_c} f_i\left(\mathbf{h}_{c,i}^{t-1}\right)\|^2\right] \\
&\quad + \gamma^2 \mathbb{E}\left[\|\frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{h}_s} f_i\left(\mathbf{h}_{s,i}^{t-1}\right)\|^2\right] \\
&= \frac{\gamma^2 \sum_{j=1}^{L} \sigma_j^2}{N} + \gamma^2 \mathbb{E}\left[\|\frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{w}} f_i\left(\mathbf{w}_i^{t-1}\right)\|^2\right]
\end{aligned} \tag{41}$$

We further note that

$$\begin{aligned}
&\mathbb{E}\left[\left\langle \nabla_{\mathbf{w}} f\left(\mathbf{w}^{t-1}\right), \mathbf{w}^t - \mathbf{w}^{t-1} \right\rangle\right] \\
&\overset{(a)}{=} -\gamma \mathbb{E}\left[\left\langle \nabla_{\mathbf{w}} f\left(\mathbf{w}^{t-1}\right), \frac{1}{N} \sum_{i=1}^{N} \mathbf{g}_i^t \right\rangle\right] \\
&\overset{(b)}{=} -\gamma \mathbb{E}\left[\left\langle \nabla_{\mathbf{w}} f\left(\mathbf{w}^{t-1}\right), \frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{w}} f_i\left(\mathbf{w}_i^{t-1}\right) \right\rangle\right] \\
&\overset{(c)}{=} -\frac{\gamma}{2} \mathbb{E}\left[\left\|\nabla_{\mathbf{w}} f\left(\mathbf{w}^{t-1}\right)\right\|^2 + \left\|\frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{w}} f_i\left(\mathbf{w}_i^{t-1}\right)\right\|^2\right. \\
&\quad \left. - \left\|\nabla_{\mathbf{w}} f\left(\mathbf{w}^{t-1}\right) - \frac{1}{N} \sum_{i=1}^{N} \nabla_{\mathbf{w}} f_i\left(\mathbf{w}_i^{t-1}\right)\right\|^2\right]
\end{aligned} \tag{42}$$

where (a) follows from $\mathbf{w}^t = \frac{1}{N} \sum_{i=1}^{N} \mathbf{w}_i^t$; (c) follows from the identity $\langle \mathbf{z}_1, \mathbf{z}_2 \rangle = \frac{1}{2}\left(\|\mathbf{z}_1\|^2 + \|\mathbf{z}_2\|^2 - \|\mathbf{z}_1 - \mathbf{z}_2\|^2\right)$ for any two vectors $\mathbf{z}_1, \mathbf{z}_2$ of the same length; (b) follows from

$$\mathbb{E}\left[\langle \nabla_{\mathbf{w}} f(\mathbf{w}^{t-1}), \frac{1}{N} \sum_{i=1}^{N} \mathbf{g}_i^t \rangle\right]$$

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.

$$= \mathbb{E}\left[\mathbb{E}\left[\langle \nabla_{\mathbf{w}} f(\mathbf{w}^{t-1}), \frac{1}{N}\sum_{i=1}^{N} \mathbf{g}_i^t \rangle | \boldsymbol{\xi}^{[t-1]}\right]\right]$$
$$= \mathbb{E}\left[\langle \nabla_{\mathbf{w}} f(\mathbf{w}^{t-1}), \frac{1}{N}\sum_{i=1}^{N} \mathbb{E}[\mathbf{g}_i^t | \boldsymbol{\xi}^{[t-1]}] \rangle\right]$$
$$= \mathbb{E}\left[\langle \nabla_{\mathbf{w}} f(\mathbf{w}^{t-1}), \frac{1}{N}\sum_{i=1}^{N} \nabla_{\mathbf{w}} f_i(\mathbf{w}_i^{t-1}) \rangle\right]$$

where the first equality follows by the law of expectations, the second equality follows because $\mathbf{w}^{t-1}$ is determined by $\boldsymbol{\xi}^{[t-1]} = [\boldsymbol{\xi}^1, \ldots, \boldsymbol{\xi}^{t-1}]$ and the third equality follows from $\mathbb{E}[\mathbf{g}_i^t | \boldsymbol{\xi}^{[t-1]}] = \mathbb{E}[\nabla F_i(\mathbf{w}_i^{t-1}; \xi_i^t) | \boldsymbol{\xi}^{[t-1]}] = \nabla f_i(\mathbf{w}_i^{t-1})$.

Substituting Eqn. (41) and Eqn. (42) into Eqn. (37), we have

$$\begin{aligned}
&\mathbb{E}\left[f\left(\mathbf{w}^t\right)\right] \\
&\leq \mathbb{E}\left[f\left(\mathbf{w}^{t-1}\right)\right] - \frac{\gamma - \gamma^2\beta}{2}\mathbb{E}\left[\left\|\frac{1}{N}\sum_{i=1}^{N}\nabla_{\mathbf{w}} f_i\left(\mathbf{w}_i^{t-1}\right)\right\|^2\right] \\
&\quad - \frac{\gamma}{2}\mathbb{E}\left[\left\|\nabla_{\mathbf{w}} f\left(\mathbf{w}^{t-1}\right)\right\|^2 + \frac{\beta\gamma^2\sum_{j=1}^{L}\sigma_j^2}{2N}\right. \\
&\quad + \frac{\gamma}{2}\mathbb{E}\left[\left\|\nabla_{\mathbf{w}} f\left(\mathbf{w}^{t-1}\right) - \frac{1}{N}\sum_{i=1}^{N}\nabla_{\mathbf{w}} f_i\left(\mathbf{w}_i^{t-1}\right)\right\|^2\right] \\
&\overset{(a)}{\leq} \mathbb{E}\left[f\left(\mathbf{w}^{t-1}\right)\right] - \frac{\gamma}{2}\mathbb{E}\left[\left\|\nabla_{\mathbf{w}} f\left(\mathbf{w}^{t-1}\right)\right\|^2 + \frac{\beta\gamma^2\sum_{j=1}^{L}\sigma_j^2}{2N}\right. \\
&\quad + \frac{\gamma}{2}\mathbb{E}\left[\left\|\nabla_{\mathbf{h}_c} f\left(\mathbf{h}_c^{t-1}\right) - \frac{1}{N}\sum_{i=1}^{N}\nabla_{\mathbf{h}_c} f_i\left(\mathbf{h}_{c,i}^{t-1}\right)\right\|^2\right] \\
&\quad + \frac{\gamma}{2}\mathbb{E}\left[\left\|\nabla_{\mathbf{h}_s} f\left(\mathbf{h}_s^{t-1}\right) - \frac{1}{N}\sum_{i=1}^{N}\nabla_{\mathbf{h}_s} f_i\left(\mathbf{h}_{s,i}^{t-1}\right)\right\|^2\right] \\
&\overset{(b)}{\leq} \mathbb{E}\left[f\left(\mathbf{w}^{t-1}\right)\right] - \frac{\gamma}{2}\mathbb{E}\left[\left\|\nabla_{\mathbf{w}} f\left(\mathbf{w}^{t-1}\right)\right\|^2\right] + \frac{\beta\gamma^2\sum_{j=1}^{L}\sigma_j^2}{2N} \\
&\quad + \mathbb{1}_{\{I>1\}} 2\beta^2\gamma^3 I^2 \sum_{j=1}^{L_c} G_j^2
\end{aligned} \tag{43}$$

where (a) follows from $0 < \gamma \leq \frac{1}{\beta}$ and (b) holds because of the following inequality (44) and (45)

$$\begin{aligned}
&\mathbb{E}\left[\|\nabla_{\mathbf{h}_c} f(\mathbf{h}_c^{t-1}) - \frac{1}{N}\sum_{i=1}^{N}\nabla_{\mathbf{h}_c} f_i(\mathbf{h}_{c,i}^{t-1})\|^2\right] \\
&= \mathbb{E}\left[\|\frac{1}{N}\sum_{i=1}^{N}\nabla_{\mathbf{h}_c} f_i(\mathbf{h}_c^{t-1}) - \frac{1}{N}\sum_{i=1}^{N}\nabla_{\mathbf{h}_c} f_i(\mathbf{h}_{c,i}^{t-1})\|^2\right] \\
&= \frac{1}{N^2}\mathbb{E}\left[\|\sum_{i=1}^{N}\left(\nabla_{\mathbf{h}_c} f_i(\mathbf{h}_c^{t-1}) - \nabla f_{\mathbf{h}_c}(\mathbf{h}_{c,i}^{t-1})\right)\|^2\right] \\
&\leq \frac{1}{N}\mathbb{E}\left[\sum_{i=1}^{N}\|\nabla_{\mathbf{h}_c} f_i(\mathbf{h}_c^{t-1}) - \nabla_{\mathbf{h}_c} f_i(\mathbf{h}_{c,i}^{t-1})\|^2\right] \\
&\leq \beta^2\frac{1}{N}\sum_{i=1}^{N}\mathbb{E}[\|\mathbf{h}_c^{t-1} - \mathbf{h}_{c,i}^{t-1}\|^2] \\
&\leq 1_{\{I>1\}} 4\beta^2\gamma^2 I^2 \sum_{j=1}^{L_c} G_j^2
\end{aligned} \tag{44}$$

where the first inequality follows by using $\|\sum_{i=1}^{N}\mathbf{z}_i\|^2 \leq N\sum_{i=1}^{N}\|\mathbf{z}_i\|^2$ for any vectors $\mathbf{z}_i$; the second inequality follows from the smoothness of each f_i by Assumption 1; and the third inequality follows from Lemma 1. Moreover, we have

$$\begin{aligned}
&\mathbb{E}\left[\|\nabla_{\mathbf{h}_s} f(\mathbf{h}_s^{t-1}) - \frac{1}{N}\sum_{i=1}^{N}\nabla_{\mathbf{h}_s} f_i(\mathbf{h}_{s,i}^{t-1})\|^2\right] \\
&\leq \beta^2\frac{1}{N}\sum_{i=1}^{N}\mathbb{E}[\|\mathbf{h}_s^{t-1} - \mathbf{h}_{s,i}^{t-1}\|^2] \overset{(a)}{=} 0
\end{aligned} \tag{45}$$

where (a) holds because the server-side sub-models are aggregated in each training round (i.e., $I = 1$). Therefore, at any training round t, the server-side sub-model of each edge device is the aggregated version of server-side sub-models.

Dividing the both sides of Eqn. (43) by $\frac{\gamma}{2}$ and rearranging terms yields

$$\begin{aligned}
&\mathbb{E}\left[\|\nabla_{\mathbf{w}} f(\mathbf{w}^{t-1})\|^2\right] \\
&\leq \frac{2}{\gamma}\left(\mathbb{E}\left[f(\mathbf{w}^{t-1})\right] - \mathbb{E}\left[f(\mathbf{w}^t)\right]\right) \\
&\quad + \frac{\beta\gamma\sum_{j=1}^{L}\sigma_j^2}{N} + 1_{\{I>1\}} 4\beta^2\gamma^2 I^2 \sum_{j=1}^{L_c} G_j^2
\end{aligned}$$

Summing over $t \in \{1, \ldots, R\}$ and dividing both sides by R yields

$$\begin{aligned}
&\frac{1}{R}\sum_{t=1}^{R}\mathbb{E}\left[\|\nabla_{\mathbf{w}} f(\mathbf{w}^{t-1})\|^2\right] \\
&\leq \frac{2}{\gamma R}\left(f(\mathbf{w}^0) - \mathbb{E}\left[f(\mathbf{w}^R)\right]\right) + \frac{\beta\gamma\sum_{j=1}^{L}\sigma_j^2}{N} \\
&\quad + 1_{\{I>1\}} 4\beta^2\gamma^2 I^2 \sum_{j=1}^{L_c} G_j^2 \\
&\overset{(a)}{\leq} \frac{2}{\gamma R}\left(f(\mathbf{w}^0) - f^*\right) \\
&\quad + \frac{\beta\gamma\sum_{j=1}^{L}\sigma_j^2}{N} + 1_{\{I>1\}} 4\beta^2\gamma^2 I^2 \sum_{j=1}^{L_c} G_j^2
\end{aligned}$$

where (a) follows because f^* is the minimum value of problem (1).

C. Proof of Theorem 2

We first conduct a functional analysis for the objective function in problem (32). Let $\Theta'(I) = \frac{2\vartheta\{aI+b\}}{\gamma I(c-4\beta^2\gamma^2 I^2 T_1)}$, where $a = T_3 + T_s^F + T_s^B + T_4$, $b = T_5 + T_6$ and $c = \varepsilon - \frac{\beta\gamma\sum_{j=1}^{L}\sigma_j^2}{N}$. Taking the first-order derivative of $\Theta'(I)$ yields

$$\frac{\partial\Theta'(I)}{\partial I} = \frac{2\vartheta}{\gamma}\frac{\Xi(I)}{(cI - 4\beta^2\gamma^2 I^3 T_1)^2} \tag{46}$$

where

$$\Xi(I) = 8a\beta^2\gamma^2 I^3 T_1 + 12b\beta^2\gamma^2 I^2 T_1 - bc. \tag{47}$$

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.

Since $\frac{\partial \Xi(I)}{\partial I} = 24\beta^2\gamma^2 I T_1 (aI + b) > 0$, $\Xi(I)$ is an increasing function of I. Considering that $\Xi(0) = -bc < 0$ and $\lim_{I \to +\infty} \Xi(I) = +\infty > 0$, there must exist I' satisfying $\Xi(I') = 0$ for $I' \in (0, +\infty)$, which can be easily obtained by classical Newton-Raphson method. Then, we see that $\frac{\partial \Theta'(I)}{\partial I} \leq 0$ for $I \in (0, I']$ and $\frac{\partial \Theta'(I)}{\partial I} \geq 0$ for $I \in (I', +\infty)$, which means that the objective function decreases and then increases with respective to I and thus reaches a minimum at $I = I'$.

Based on constraint C4 and the characteristics of the objective function analyzed above, it is obvious that the optimal value I^* only exists on both sides of I', i.e.,

$$I^* = \begin{cases} 1 & I' \leq 1 \\ \arg\min_{I \in \{\lfloor I' \rfloor, \lceil I' \rceil\}} \Theta'(I) & I' > 1, \end{cases} \tag{48}$$

where $\lfloor \cdot \rfloor$ and $\lceil \cdot \rceil$ denote floor and ceiling operations.

REFERENCES

[1] IDC. (Nov. 2019). *How You Contribute To Today's Growing DataSphere and Its Enterprise Impact*. [Online]. Available: https://blogs.idc.com/2019/11/04/how-you-contribute-to-todays-growing-datasphere-and-its-enterprise-impact/

[2] K. B. Letaief, Y. Shi, J. Lu, and J. Lu, "Edge artificial intelligence for 6G: Vision, enabling technologies, and applications," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 1, pp. 5–36, Jan. 2022.

[3] G. Zhu, D. Liu, Y. Du, C. You, J. Zhang, and K. Huang, "Toward an intelligent edge: Wireless communication meets machine learning," *IEEE Commun. Mag.*, vol. 58, no. 1, pp. 19–25, Jan. 2020.

[4] Z. Fang, Z. Lin, Z. Chen, X. Chen, Y. Gao, and Y. Fang, "Automated federated pipeline for parameter-efficient fine-tuning of large language models," 2024, *arXiv:2404.06448*.

[5] Z. Lin, L. Wang, J. Ding, B. Tan, and S. Jin, "Channel power gain estimation for terahertz Vehicle-to-Infrastructure networks," *IEEE Commun. Lett.*, vol. 27, no. 1, pp. 155–159, Jan. 2023.

[6] S. Hu, Z. Fang, Y. Deng, X. Chen, and Y. Fang, "Collaborative perception for connected and autonomous driving: Challenges, possible solutions and opportunities," 2024, *arXiv:2401.01544*.

[7] Z. Fang et al., "IC3M: In-car multimodal multi-object monitoring for abnormal status of both driver and passengers," 2024, *arXiv:2410.02592*.

[8] X. Chen, G. Zhu, H. Ding, L. Zhang, H. Zhang, and Y. Fang, "End-to-end service auction: A general double auction mechanism for edge computing services," *IEEE/ACM Trans. Netw.*, vol. 30, no. 6, pp. 2616–2629, Dec. 2022.

[9] X. Liu, Z. Yan, Y. Zhou, D. Wu, X. Chen, and J. H. Wang, "Optimizing parameter mixing under constrained communications in parallel federated learning," *IEEE/ACM Trans. Netw.*, vol. 31, no. 6, pp. 2640–2652, Dec. 2023.

[10] Y. Deng, X. Chen, G. Zhu, Y. Fang, Z. Chen, and X. Deng, "Actions at the edge: Jointly optimizing the resources in multi-access edge computing," *IEEE Wireless Commun.*, vol. 29, no. 2, pp. 192–198, Apr. 2022.

[11] X. Chen, G. Zhu, Y. Deng, and Y. Fang, "Federated learning over multihop wireless networks with in-network aggregation," *IEEE Trans. Wireless Commun.*, vol. 21, no. 6, pp. 4622–4634, Jun. 2022.

[12] H. B. Mcmahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera Y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, Jan. 2017, pp. 1–13.

[13] J. Konečny, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," *arXiv:1610.05492*, Oct. 2016.

[14] Z. Lin, Z. Chen, Z. Fang, X. Chen, X. Wang, and Y. Gao, "FedSN: A federated learning framework over heterogeneous LEO satellite networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 3, pp. 1293–1307, Mar. 2025.

[15] M. Hu, J. Zhang, X. Wang, S. Liu, and Z. Lin, "Accelerating federated learning with model segmentation for edge networks," *IEEE Trans. Green Commun. Netw.*, vol. 9, no. 1, pp. 242–254, Mar. 2025.

[16] Z. Lin, G. Qu, Q. Chen, X. Chen, Z. Chen, and K. Huang, "Pushing large language models to the 6G edge: Vision, challenges, and opportunities," 2023, *arXiv:2309.16739*.

[17] G. Team et al., "Gemini: A family of highly capable multimodal models," 2023, *arXiv:2312.11805*.

[18] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split learning for health: Distributed deep learning without sharing raw patient data," 2018, *arXiv:1812.00564*.

[19] Z. Lin, G. Qu, X. Chen, and K. Huang, "Split learning in 6G edge networks," *IEEE Wireless Commun.*, vol. 31, no. 4, pp. 170–176, Aug. 2024.

[20] Z. Lin et al., "Hierarchical split federated learning: Convergence analysis and system optimization," 2024, *arXiv:2412.07197*.

[21] C. Thapa, M. A. P. Chamikara, S. Camtepe, and L. Sun, "SplitFed: When federated learning meets split learning," in *Proc. AAAI*, vol. 36, Jun. 2022, pp. 8485–8493.

[22] Huawei, *NET4AI: Supporting AI As a Service 6G*. Cambridge, U.K.: Cambridge Univ. Press, Sep. 2022.

[23] L. You, S. Liu, T. Wang, B. Zuo, Y. Chang, and C. Yuen, "AiFed: An adaptive and integrated mechanism for asynchronous federated data mining," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 9, pp. 4411–4427, Sep. 2024.

[24] K. Wei et al., "Federated learning with differential privacy: Algorithms and performance analysis," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3454–3469, 2020.

[25] S. Wang et al., "Adaptive federated learning in resource constrained edge computing systems," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 6, pp. 1205–1221, Jun. 2019.

[26] X. Wu, F. Huang, Z. Hu, and H. Huang, "Faster adaptive federated learning," in *Proc. AAAI*, vol. 37, Jun. 2023, pp. 10379–10387.

[27] S. L. Smith, P.-J. Kindermans, and Q. V. Le, "Don't decay the learning rate, increase the batch size," in *Proc. ICLR*, Feb. 2018, pp. 1–21.

[28] H. Yu and R. Jin, "On the computation and communication complexity of parallel SGD with dynamic batch sizes for stochastic non-convex optimization," in *Proc. ICML*, May 2019, pp. 7174–7183.

[29] T. Xiang et al., "Federated learning with dynamic epoch adjustment and collaborative training in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 1–16, Jun. 2023.

[30] W. Wu et al., "Split learning over wireless networks: Parallel design and resource management," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 4, pp. 1051–1066, Apr. 2023.

[31] S. Wang, X. Zhang, H. Uchiyama, and H. Matsuda, "HiveMind: Towards cellular native machine learning model splitting," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 2, pp. 626–640, Feb. 2021.

[32] Z. Lin et al., "Efficient parallel split learning over resource-constrained wireless edge networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9224–9239, Oct. 2024.

[33] S. Pal et al., "Server-side local gradient averaging and learning rate acceleration for scalable split learning," 2021, *arXiv:2112.05929*.

[34] Y. Liao, Y. Xu, H. Xu, Z. Yao, L. Huang, and C. Qiao, "ParallelSFL: A novel split federated learning framework tackling heterogeneity issues," in *Proc. 30th Annu. Int. Conf. Mobile Comput. Netw.*, Dec. 2024, pp. 845–860.

[35] J. Guo, J. Wu, A. Liu, and N. N. Xiong, "LightFed: An efficient and secure federated edge learning system on model splitting," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2701–2713, Nov. 2022.

[36] D. Pasquini, G. Ateniese, and M. Bernaschi, "Unleashing the tiger: Inference attacks on split learning," in *Proc. CCS*, vol. 2021, Nov. 2021, pp. 2113–2129.

[37] F. Zhou and G. Cong, "On the convergence properties of a K-step averaging stochastic gradient descent algorithm for nonconvex optimization," in *Proc. 27th Int. Joint Conf. Artif. Intell.*, Jul. 2018, pp. 3219–3227.

[38] H. Yu, R. Jin, and S. Yang, "On the linear speedup analysis of communication efficient momentum SGD for distributed non-convex optimization," in *Proc. 36th Int. Conf. Machine Learning, (ICML)*, 2019, pp. 7184–7193.

[39] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. ICLR*, Apr. 2019, pp. 5132–5143.

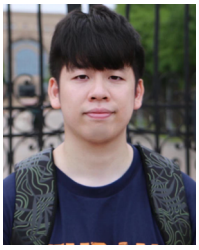
[40] Y. Zhang, J. C. Duchi, and M. J. Wainwright, "Communication-efficient algorithms for statistical optimization," in *Proc. IEEE 51st IEEE Conf. Decis. Control (CDC)*, Dec. 2012, pp. 1–9.

[41] X. Lian, C. Zhang, H. Zhang, C.-J. Hsieh, W. Zhang, and J. Liu, "Can decentralized algorithms outperform centralized algorithms? A case study for decentralized parallel stochastic gradient descent," in *Proc. NIPS*, vol. 30, Dec. 2017, pp. 5330–5340.

[42] H. Mania, X. Pan, D. Papailiopoulos, B. Recht, K. Ramchandran, and M. I. Jordan, "Perturbed iterate analysis for asynchronous stochastic optimization," *SIAM J. Optim.*, vol. 27, no. 4, pp. 2202–2229, Jan. 2017.

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.

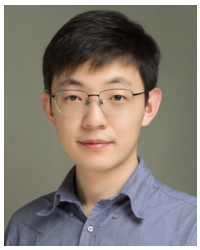
[43] T. Lin, S. U. Stich, K. K. Patel, and M. Jaggi, "Don't use large mini-batches, use local SGD," in *Proc. 8th Int. Conf. Learn. Represent. (ICLR)*, Apr. 2020, pp. 1–15.
[44] Z. Yang, M. Chen, W. Saad, C. S. Hong, and M. Shikh-Bahaei, "Energy efficient federated learning over wireless communication networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 3, pp. 1935–1949, Mar. 2020.
[45] A. Tirumala, (1999)*IPERF: The TCP/UDP Bandwidth Measurement Tool*. [Online]. Available: http://dast.nlanr.net/Projects/Iperf
[46] Z. Zhang, Z. Gao, Y. Guo, and Y. Gong, "Scalable and low-latency federated learning with cooperative mobile edge networking," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 812–822, Jan. 2022.
[47] X. Deng et al., "Low-latency federated learning with DNN partition in distributed industrial IoT networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 3, pp. 755–775, Mar. 2023.
[48] W. Shi, S. Zhou, Z. Niu, M. Jiang, and L. Geng, "Joint device scheduling and resource allocation for latency constrained wireless federated learning," *IEEE Trans. Wireless Commun.*, vol. 20, no. 1, pp. 453–467, Sep. 2020.
[49] W. Xia, W. Wen, K.-K. Wong, T. Q. Quek, J. Zhang, and H. Zhu, "Federated-learning-based client scheduling for low-latency wireless communications," *IEEE Wireless Commun.*, vol. 28, no. 2, pp. 32–38, Apr. 2021.
[50] W. Dinkelbach, "On nonlinear fractional programming," *Manag. Sci.*, vol. 13, no. 7, pp. 492–498, Mar. 1967.
[51] D. Yue and F. You, "A reformulation-linearization method for the global optimization of large-scale mixed-integer linear fractional programming problems and cyclic scheduling application," in *Proc. Amer. Control Conf.*, Jun. 2013, pp. 3985–3990.
[52] R. G. Ródenas, M. L. López, and D. Verastegui, "Extensions of Dinkelbach's algorithm for solving non-linear fractional programming problems," *Top*, vol. 7, no. 1, pp. 33–70, Jun. 1999.
[53] P. Tseng, "Convergence of a block coordinate descent method for nondifferentiable minimization," *J. Optim. Theory Appl.*, vol. 109, no. 3, pp. 475–494, Jun. 2001.
[54] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," Toronto, ON, Canada, 2009.
[55] G. Zhu, Y. Wang, and K. Huang, "Broadband analog aggregation for low-latency federated edge learning," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 491–506, Jan. 2019.
[56] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. ICLR*, Jan. 2015, pp. 1–14.
[57] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
[58] Z. Wang, H. Xu, Y. Xu, Z. Jiang, and J. Liu, "CoopFL: Accelerating federated learning with DNN partitioning and offloading in heterogeneous edge computing," *Comput. Netw.*, vol. 220, Jan. 2023, Art. no. 109490.
[59] W. J. R. Moore, F. Esposito, and M. A. Zuluaga, "DTS: A simulator to estimate the training time of distributed deep neural networks," in *Proc. 30th Int. Symp. Model., Anal., Simul. Comput. Telecommun. Syst. (MASCOTS)*, Oct. 2022, pp. 17–24.
[60] Y. Yoo and S. Jung, "Modeling forecast errors for microgrid operation using Gaussian process regression," *Sci. Rep.*, vol. 14, no. 1, p. 2166, Jan. 2024.



Zheng Lin received the B.Eng. degree in electronic information engineering from Fuzhou University in 2020 and the M.Eng. degree in electrical and computer engineering from Fudan University in 2023. He is currently pursuing the Ph.D. degree with the Department of Electrical and Electronic Engineering, The University of Hong Kong, Hong Kong, China. His research interests include wireless networking, edge intelligence, and distributed machine learning. He was awarded Fudan Outstanding Graduate and Shanghai Outstanding Graduate in 2023. He serves as a Young Professional for the IEEE Vehicular Technology Society Ad Hoc Committee and a Program Committee Member for several top-tier international conferences, such as ICLR, ICML, NeurIPS, ACM MM, and AAAI.

Guanqiao Qu (Graduate Student Member, IEEE) received the B.E. and M.E. degrees from Harbin Institute of Technology (HIT), Harbin, China, in 2020 and 2022, respectively. He is currently pursuing the Ph.D. degree with the Department of Electrical and Electronic Engineering, The University of Hong Kong (HKU), Hong Kong, China. His research interests include wireless communications, networking, edge intelligence, and machine learning.

Wei Wei received the B.S. degree in electronics engineering from Shandong University, Jinan, China, in 2020, and the M.S. degree (Hons.) in electronic and information engineering from Tsinghua University, Beijing, China, in 2023. She is currently pursuing the Ph.D. degree with the Department of Electrical and Electronic Engineering, The University of Hong Kong, Hong Kong, China. Her research interests include edge computing and multi-agent reinforcement learning.



Xianhao Chen (Member, IEEE) received the B.Eng. degree in electronic information from Southwest Jiaotong University in 2017 and the Ph.D. degree in electrical and computer engineering from the University of Florida in 2022. He is currently an Assistant Professor at the Department of Electrical and Electronic Engineering, The University of Hong Kong, where he directs the Wireless Information and Intelligence (WILL) Laboratory. His research interests include wireless networking, edge intelligence, and machine learning. He serves as a TPC member for several international conferences. He received the Early Career Award from the Research Grants Council (RGC) of Hong Kong in 2024, the ECE Graduate Excellence Award for research from the University of Florida in 2022, and the ICCC Best Paper Award in 2023. He serves as an Associate Editor for *ACM Computing Surveys*.

Kin K. Leung (Life Fellow, IEEE) received the B.S. degree from the Chinese University of Hong Kong, and the M.S. and Ph.D. degrees from the University of California, Los Angeles. He worked at AT&T Bell Labs and its successor companies in New Jersey from 1986 to 2004. Since then, he has been the Tanaka Chair Professor at Imperial College, London, U.K. He is a Co-Director of School of Convergence Science in Space, Security and Telecommunications at Imperial. His current research focuses on machine learning for communication, computing and quantum networks. He also works on multi-antenna systems for wireless networks. He is a Fellow of Royal Academy of Engineering, IET Fellow, and member of Academia Europaea. He received the Royal Society Wolfson Research Merits Award, and the Bell Labs Distinguished Member of Technical Staff Award. Jointly with his collaborators, he won the IEEE ComSoc Leonard G. Abraham Prize, IEEE ComSoc Best Survey Paper Award, U.S.-U.K. Science and Technology Stocktake Award, Lanchester Prize Honorable Mention Award, and several best conference paper awards. He chaired the IEEE ComSoc Fellow Evaluation Committee (2012-2015) and has served as editor for ten IEEE and *ACM journals*. He is now an editor for the *ACM Computing Survey and International Journal on Sensor Networks*.

Authorized licensed use limited to: Queen's University. Downloaded on March 16,2026 at 15:59:50 UTC from IEEE Xplore. Restrictions apply.